

Technical Appendix: Synthesizing Staged Updates between Verified Controllers

Anonymous Author(s)

Method and model names. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) is compared with Dynamic Update Controller Synthesis (DUCS), generalized reactivity of rank 1 (GR(1)) updating, and directed controller synthesis (DCS), including its on-the-fly variant (OTF-DCS). On-the-fly is abbreviated OTF. The Modal Transition System Analyser (MTSA) processes Finite State Processes (FSP). Labeled transition systems (LTSs), modal transition systems (MTSs) and deterministic finite automata (DFAs) represent behavior. PC1 and PC2 name the one-arm and two-arm production-cell inputs; GSM names the communication input. DB-Rolling is the database-inspired rolling-update family.

Requirement and result labels. Uncontrollable (UC) events cannot be disabled by a controller. Residual soundness (RS) concerns declared prior histories; activation-scoped interpretation (ACT) starts an invariant at activation. OLD, NEW and UPD label old, new and update-interval requirements. WIN means a completing policy exists; LOSS means no policy covers every entry. TO denotes timeout, OOM an out-of-memory failure, and N/M a case not measured. Research questions are labeled RQ1–RQ3. Other uppercase model states and event names are identifiers retained from the supplied inputs.

Implementation and measurement terms. The Java Virtual Machine (JVM) runs a Java archive (JAR). JavaScript Object Notation (JSON) and comma-separated values (CSV) are data formats. Last-in, first-out (LIFO) and first-in, first-out (FIFO) describe queue orders. Resident set size (RSS) measures process-resident memory; garbage collection (GC) concerns managed memory. Artificial intelligence (AI) use is disclosed separately. GiB and MiB denote gibibytes and mebibytes.

All appendix and S-section references point to the accompanying Supplementary Material for this revision. The public release [4] supplies the matching manuscript, supplement and source alongside the unchanged implementation and saved experiment outputs (main-paper Data Availability).

Read A–C for the contract, synthesis and complete Cell/Policy examples; D for the complete granularity models, counterexamples and serialization proof; E–H for requirement meanings, execution and endpoint interfaces; I–J for existing update interfaces; and K–Q for case, exploration and evaluation details. The full lazy procedure is in A; cross-references connect its theorem to the main paper. The current validation summary appears at the start of S4, with historical initialization diagnostics clearly separated. The ordering brings the granularity argument next to the examples rather than placing it among auxiliary evaluation records.

A Source Histories and Synthesis Correctness

Source histories without a global game. Fix the eligible local contract of Section 3. A history is $\ell = z(a_1, o_1) \cdots (a_n, o_n)$, starting at a reachable old closed-loop tuple z ; o_j records the realized physical successor tuple. For each prefix, reconstruct installed versions $v_i(\ell)$ and local states $s_i(\ell)$ by the recurrences below. Let $D(\ell)$ be its executed update commands. Initially $D = \emptyset$, every version is old and local states are those in z . Old monitor memory is inherited from z , each interval monitor starts at its supplied initializer, and every new monitor is inactive. The old endpoint controller imposes no restrictions after entry.

Enabled local events and outcomes. For an ordinary label a , let $J_a(\ell) = \{i : a \in \Sigma_{e_i}^{v_i(\ell)}\}$. It is enabled exactly when $J_a(\ell) \neq \emptyset$ and every $i \in J_a(\ell)$ has a nonempty local a -successor set. Every tuple choosing one such successor per participant, and retaining all other local states, is an outcome. This test ignores monitors, safety and targets. The prefix is UC-quiescent exactly when no enabled ordinary label is uncontrollable, including labels with unsafe monitor outcomes. A command a may occur only at a UC-quiescent prefix, with $a \notin D(\ell)$ and all its declared predecessors in $D(\ell)$.

An ordinary event preserves versions and D . A transfer ρ_i additionally requires $v_i = \text{old}$ and chooses any $s'_i \in g_i(s_i)$, changes only that installed component to (new, s'_i) , and adds ρ_i to D . An empty transfer image permits no extension. A stop or start leaves all physical states unchanged and adds itself to D ; start additionally requires the current version-tagged tuple in its initializer domain. Every enabled outcome yields a source extension, even if it violates a monitor. Thus monitors test behavior rather than disabling physical events.

Memory, boundaries and source objectives. Monitor memory follows a deterministic fold on the history. Ordinary events and transfers advance every active monitor by its transition function. A stop makes its named old monitor inactive without letting it observe that stop, and advances all other active monitors once. A start advances previously active monitors once, then installs the named new monitor's initializer value; the new monitor does not consume start again. Other inactive memories remain inactive. The executed-command set makes every boundary one-shot. A prefix is safe when every active memory is outside its error set.

Handover is allowed precisely at safe UC-quiescent prefixes with every command executed and with the physical tuple and new-monitor memories matching a supplied target; the selected target also supplies its controller state. A source policy may depend on the entire history and may disable only controllable labels. Every enabled uncontrollable, and every outcome of every permitted label, remains possible. A policy wins if all its prefixes from every old entry are safe, no non-handover prefix is a dead end, and every maximal history reaches handover after finitely many events. Neither fairness nor a choice of local outcomes is available to the policy. These operational clauses precede the global-state construction and are the source obligations used in Proposition 3.1.

Source histories describe supplied monitor behavior. They do not infer the semantic activation-history sets H_r in RS or prove those sets complete for an application. The scope-check populations and their limits are unchanged. The main paper states Proposition 3.1 for these source operations.

Proof of Proposition 3.1.

Proof. Write H for the source histories specified above and define

$$F(z(a_1, o_1) \cdots (a_n, o_n)) = (\xi_n, \mu_n, \Sigma_{\text{upd}} \setminus D_n),$$

where ξ_n, μ_n, D_n are obtained by the local recurrences. At length zero this is exactly $\text{Init}(z)$, so the images of initial histories are exactly Q_0 . Distinct old-controller states may have the same image; old control has been released, so this identification removes no subsequent source restriction.

We show the one-step property: a source history h permits label a with realized component outcome o iff the generated game at $F(h)$ permits that label with the same component outcome, and the resulting source extension has the resulting game state as its image. For an ordinary label, the participating set $J_a(h)$ is determined by the current versions. Both definitions require a nonempty participating set and every participant to have a transition and include their full Cartesian product, retaining nonparticipants. Both then apply the same deterministic transition to every active monitor. For ρ_i , both definitions require UC quiescence, an unexecuted command with all predecessors executed, an old component and a successor in $g_i(s_i)$. Both change only that component, advance active monitors and remove that command from the pending set. For a stop, both make the named monitor inactive without observing its own boundary and advance all other active monitors once. For a start, both require the initializer domain, advance previously active monitors once and install the named monitor's

initializer without a second observation. The executed/pending sets are complements, so one-shot and precedence tests coincide.

UC quiescence in each case is computed from every enabled ordinary UC label before any monitor-safety filtering. Thus even unsafe UC outcomes are preserved. Monitor safety, completion of all commands and equality to each projected loadable target are read from identical coordinates. Consequently F preserves and reflects safety, handover eligibility and every compatible full target. Histories with the same image have the same extensions up to their realized physical outcomes and the same resulting images.

Map a source history to the entire game path of its prefix images, retaining its labels and component outcomes. Repeated application of the one-step property maps every finite source history to a game path. Conversely choose an old tuple witnessing any first state in Q_0 , then lift every edge using its recorded component outcome and the one-step property. This produces a source history for every finite game path and, by the same recurrence, for every infinite path. Different initial witnesses can give different histories but admit the same extensions and objectives.

A game history-dependent policy lifts to a source policy by applying it to this image path. Conversely, for each initial game state choose one old-tuple witness and consistently lift each subsequent path from that witness; apply the source policy to the lifted history. This defines a game history-dependent policy. In either direction enabled labels, all uncontrollable choices and every permitted-label outcome are preserved. Every path of the translated game policy lifts to a source-policy path from its chosen initial witness; every path of the lifted source policy projects to a game-policy path. A winning policy therefore acquires no unsafe prefix, non-goal deadlock or infinite avoiding path under either translation. Hence a winning source policy exists iff a winning game history-dependent policy exists. The finite-game memoryless-policy lemma below supplies a state-dependent winner with a strict rank, completing the proposition. This is an operational correspondence for the supplied contract, not a proof of frontend correctness or application-history completeness. $\square$

Lemma (memoryless policies suffice). If any history-dependent policy wins this finite game, a state-based policy wins. *Proof.* At a safe non-goal, a proving set is the full nonempty UC successor union if UC is enabled, or one complete nonempty controllable bucket otherwise. Starting with goals, attractor layers add safe states whose proving sets lie in previous layers. Choosing those sets gives a state-based policy with decreasing layer index. Outside the fixed point, every controller choice has an outcome outside, or the state is unsafe or a non-goal deadlock. The environment can keep selecting such outcomes after every history, defeating any history-dependent policy.

Lemma (monitor intervals). Along every update prefix, each active monitor is its inherited or initialized state advanced over exactly the labels in its declared active interval. *Proof.* Entry inherits old states and initializes interval monitors. Ordinary events and transfers advance all active monitors once. Stop removes its monitor before observation, while others advance. Start advances existing monitors and installs the new one after its boundary. Pending commands enforce one-shot execution and consistent versions/activity. Induction gives the stated interval; safety makes it non-error, and goal matching preserves new-monitor states into the endpoint.

Maximal runs and ranks. Let $R = \text{Reach}_\sigma(Q_0)$ be computed without expanding goals. A maximal run is infinite, or finite and ending at a goal or a state with no policy successor. Assume R is finite, all its states are safe, goals are terminal, and every non-goal has a successor. If the reachable non-goal subgraph has a cycle, first follow a path from a root to that cycle and then repeat it. Every edge is policy-permitted, including its selected nondeterministic outcome, so this is an infinite maximal run avoiding goals. Universal finite completion excludes it without invoking fairness.

Conversely, if that subgraph is acyclic, no path can visit a non-goal twice. Every path therefore has at most $|R \setminus Q_{\text{goal}}|$ non-goal edges before ending. A non-goal endpoint is impossible by the successor premise. Thus every maximal run ends at a goal. Define $\text{rk}(q)$ as the maximum length of a goal-ending

path from q , and set goal ranks to zero. Such a path exists because the graph is finite, acyclic off goals, and has no non-goal sinks. For each edge $q \rightarrow q'$ from a non-goal, prefixing a longest path from q' proves $\text{rk}(q) \geq 1 + \text{rk}(q')$. This gives a positive natural-number rank at non-goals. Finally, a strict decrease around a cycle would give $\text{rk}(q) > \text{rk}(q)$, a contradiction. These implications establish all three equivalences. Without the successor premise, a non-goal sink is an acyclic counterexample.

A *bucket* is $\text{Post}(q, a)$; empty means disabled. Disabling controls at a UC-enabled state removes runs without creating deadlock, preserving winning (*UC priority*, Lemma A.1). With quiescent goals and no fairness or controller priority, permitting commands while UC is enabled therefore adds no winning entries. Changing fairness or priority changes the game. The proof establishes memoryless sufficiency.

A.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [6]. V contains discovered states, X those with all UC candidates queried, and W proved winners. *Promote* adds an unproved safe state in X to W when its nonempty UC successor set lies in W or, at quiescence, one complete nonempty control bucket lies in W . It retains that proving set and assigns rank one above its maximum successor rank. Predecessors are then reconsidered. Lazy queries controls incrementally, keeping unfinished cursors in *Retry*. Failure requires exhausted candidates and no further promotion. Extraction retains lower-rank controls and matching endpoint targets.

Algorithm 1: Lazy

Input: Typed update contract (Section 3)
Output: $(\sigma, \text{rk}, \kappa)$ or (no solution, L)

- 1 $V \leftarrow Q_0$; $X, \text{Retry} \leftarrow \emptyset$; $W \leftarrow V \cap Q_{\text{goal}}$;
- 2 $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$; set $\text{rk}(q) \leftarrow 0$ for $q \in W$;
- 3 **while true do**
- 4 Add queued goals to W once with rank zero; exhaust *Promote*, retaining witnesses/ranks, removing winners from *Retry* and propagating predecessors;
- 5 **if** $Q_0 \subseteq W$ **then return** $(\sigma, \text{rk}|_{\text{Reach}_\sigma(Q_0)}, \kappa)$ constructed from the goal witnesses and retained lower-rank choices;
- 6 **if** $\text{Open} \neq \emptyset$ **then**
- 7 Remove q from *Open*, add it to X , and query all uncontrollable candidates;
- 8 Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or exhaustion;
- 9 Insert q into *Retry* iff $q \notin W$ and its cursor has remaining candidates;
- 10 **else**
- 11 **if** $\text{Retry} = \emptyset$ **then return** (no solution, $L \leftarrow V \setminus W$);
- 12 Remove q from *Retry*; query to the next nonempty bucket or exhaustion; reinsert iff $q \notin W$ and its cursor has remaining candidates;
- 13 Add every queried successor to V ; queue newly discovered goals for rank zero and safe non-goals for *Open* (unsafe states stay outside W and *Open*);

Lemma A.1 (UC priority). *Disabling controllable events at UC-enabled states preserves the existence of a policy for finite safe completion.*

Proof. A winning policy must handle every UC outcome. Disabling optional controls removes runs and creates no deadlock while a UC event remains enabled. Every surviving maximal run was already a maximal run of that policy, so it still completes safely. Conversely, the reduced policy is admissible in the original game because controllers may disable those events. $\square$

Lemma A.2 (Search coverage). *At loop boundaries, every state in X has all UC candidates queried; every expanded unproved quiescent safe non-goal with candidates left is in *Retry*; and every discovered safe non-goal outside X is in *Open*.*

Proof. Initialization at entry states establishes coverage. Discovery queues each new safe non-goal; only expansion removes it, after querying all UC candidates. At quiescence, expansion and retries consume the cursor through a nonempty bucket or exhaustion, retaining every nonwinner with candidates left. Promotion removes only winners. These are the only changes to those sets; finite propagation drains before either return. $\square$

The four exploration invariants. For the partial table $\mathcal{D}$, queried-empty buckets are recorded by candidate consumption even though they are absent from $\text{dom } \mathcal{D}$. Each stored nonempty bucket is exact and contains all modeled outcomes. We state invariants at loop boundaries, after discovery queues and the finite reverse-edge worklist have been updated.

1. **Uncontrollable completeness.** On first expansion of q , all uncontrollable candidates are consumed before promotion can use the state. No candidate is added later: the finite event set is determined by q . Therefore every $q \in X$ has its complete uncontrollable successor set known. The transient insertion into X inside the expansion block is not a return point.
2. **Controllable exhaustion at failure.** A safe non-goal state with no uncontrollable has a cursor over all controllable candidates. Expansion consumes it to the next nonempty bucket or exhaustion. If a nonwinner has candidates remaining, it is in Retry. Each retry consumes further candidates. The failure return is reached only after both queues and the propagation worklist are empty. Hence every such state in $V \setminus W$ has exhausted its cursor. Removing an entry when its state becomes winning cannot remove a losing candidate from this argument.
3. **Monotone winning evidence.** A promoted state either has all uncontrollable outcomes at smaller retained ranks, or has a complete selected controllable bucket at smaller ranks and no uncontrollable. All those buckets are exact; growing $\mathcal{D}$ adds other controllable choices rather than new outcomes to an already stored choice. The policy disables those additional controllables. Existing rank-decreasing witness edges consequently remain a valid certificate. Goals are recognized using modeled uncontrollable enabledness before terminalization, so an unqueried uncontrollable cannot invalidate rank zero.
4. **Open-state coverage.** Each newly discovered safe non-goal is inserted once in Open. It is removed only for first expansion, when it enters X . Thus $V \cap \text{Safe} \setminus (X \cup Q_{\text{goal}}) \subseteq \text{Open}$. Unsafe states and goals are discovered terminals and are handled separately.

The initial root insertion establishes these properties. Expansion, successor discovery, candidate consumption, and promotion preserve them as shown above; no other operation changes their relevant sets.

Attractor layers and retained ranks. Let $W_0 = V \cap Q_{\text{goal}}$. At the next layer a safe expanded state joins if its nonempty uncontrollable union is contained in the preceding layers, or if it has no uncontrollable and one complete nonempty controllable bucket contained there. Induction on these layers proves the existence of a winning policy for the current table. These minimum layer indices are analytical, not the implementation's retained certificate ranks. On first promotion the implementation assigns one plus the largest rank of the complete chosen successor set (all uncontrollable outcomes, or one winning controllable choice); goals have rank zero. It never lowers that assigned rank. Every stored choice used for promotion remains complete and rank-decreasing. Extraction may choose another stored controllable set only if all its successors also have smaller retained ranks. Goals satisfy the target and quiescence checks. Invariant I3 therefore preserves a valid certificate as exploration continues without claiming rank minimality.

The following proof establishes termination, soundness and completeness. The invariants above give its search-coverage and rank premises for the supplied successor and goal rules. A winning policy can be normalized by removing controls at UC-enabled states and retaining one winning control elsewhere; all UC outcomes and non-goal successors remain.

Proof of Theorem 4.1.

Proof. Termination. The finite input induces finitely many states and candidates. Each state is discovered and expanded at most once; each retry consumes a new candidate, and each promotion adds a state to W once. Reverse incidences are processed on those promotions, so every worklist drain is finite. Eventually all entries win or both exploration queues empty.

Soundness. Induction on promotions gives complete successor sets with smaller retained ranks: expansion supplies all uncontrollables, or the selected nonempty controllable bucket supplies all outcomes. A queried bucket never changes, and ranks are fixed on first promotion, so this evidence persists. If all entries win, their policy closure is safe and every non-goal has a successor. Strict decrease of the natural-number rank excludes infinite runs; nonempty successors exclude non-goal sinks. Thus every maximal run reaches a goal within its initial rank many events. Goal recognition uses modeled uncontrollable quiescence and supplies each compatible $\kappa(q)$.

The monitor-interval lemma above supplies the interval invariant; goal matching preserves components and new-monitor states into the verified endpoint.

Completeness. At failure, promotion has reached its fixed point. An entry lies in $L = V \setminus W$, and L contains no goal. By Lemma A.2, empty Open ensures every safe non-goal in L was expanded. With uncontrollables, complete expansion and exhausted propagation imply an uncontrollable outcome in L ; otherwise promotion would apply. Without uncontrollables, empty Retry gives every enabled controllable bucket, each with an outcome in L for the same reason. Against any policy, the environment keeps choosing these outcomes; permitting no event produces a deadlock. The resulting run reaches an unsafe state, deadlocks outside a goal, or remains in L forever. Each contradicts winning. Hence a winning policy rules out failure; termination then forces success.

The losing-region strategy defeats history-dependent policies after every history; success supplies a ranked state-based policy. Lemma A.1 preserves winning in the original supplied game. $\square$

A.2 Checking Returned Certificates

Checking the returned evidence. A WIN checker reconstructs the reachable policy domain $D_\sigma = \text{Reach}_\sigma(Q_0)$ from all roots and complete Post buckets and requires it to equal the rank domain. It verifies safety, enabledness of every retained action, nonempty successors outside goals, closure in the rank domain, and strict rank decrease for every outcome. At each goal it checks rank zero, physical uncontrollable quiescence, membership of $\kappa(q)$ in Z_{load} , and component/new-monitor equality. The map κ is defined exactly on $D_\sigma \cap Q_{goal}$; omitted policy entries mean the empty set. The maximal-run lemma then proves the model-level claim without reproducing the synthesizer’s exploration order.

A LOSS checker verifies a nonempty intersection with the roots, no goal in L , and the obstruction conditions of that proof. Unsafe states are terminal losses. At a safe uncontrollable-free deadlock the universal condition on enabled controllables is vacuous, correctly certifying a loss. A safe uncontrollable-free state with choices requires inspecting every enabled candidate, so checking LOSS may cost as much as the corresponding exhaustive search. These certify search relative to the successor and endpoint predicates [18]. The checks trust the shared parser, successor, monitor, transfer, and endpoint/goal semantics; the separately implemented explicit and raw-LTS checks in the fixed correctness campaign (main-paper RQ2) provide additional, scoped cross-checks.

Promotion cases. Figure A1 displays both promotion cases. Every UC outcome must win when UC is enabled. At quiescence one control bucket suffices, but all its nondeterministic outcomes must win; other control buckets may remain unqueried.

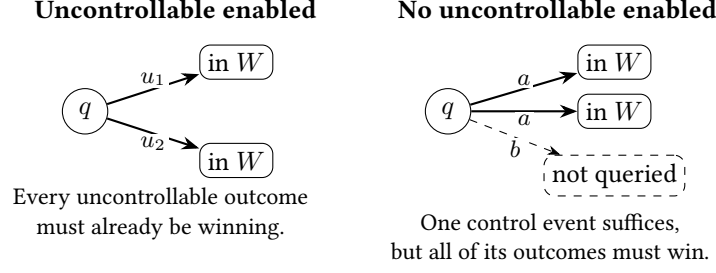


Figure A1: Safe promotion to W . The two a edges are nondeterministic outcomes of one control event, not two selectable actions. Other control buckets may be postponed.

B Complete Cell Contract and Its Two Completing Paths

Table A1 gives all component, endpoint-controller and monitor transitions, including the conventions for absent edges.

Table A1: Complete Cell components, controllers and monitors. o/n denote old/new versions; h/e mean holding/empty. Parentheses mark initial states. m_A, m_B move to A, B ; j inspects at B .

Object	States (initial)	Ordinary transitions
A^o, A^n	$h, e (h)$	$h \xrightarrow{m_B} e, e \xrightarrow{m_A} h$
B^o	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e$
B^n	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e, h \xrightarrow{j} h$
C^o	$c_o (c_o)$	$c_o \xrightarrow{m_A, m_B} c_o$
C^m	$c_0, c_1 (c_0)$	$c_0 \xrightarrow{m_A, j} c_0, c_0 \xrightarrow{m_B} c_1, c_1 \xrightarrow{j} c_0, c_1 \xrightarrow{m_B} c_1$
$t_{r_{\text{one}}}$	$A, B, \text{err} (A)$	$A \xrightarrow{m_B} B, B \xrightarrow{m_A} A, A \xrightarrow{m_A} \text{err}, B \xrightarrow{m_B} \text{err}$
$t_{r_{\text{old}}}$	$k, \text{err} (k)$	$k \xrightarrow{j} \text{err}$
$t_{r_{\text{inspect}}}$	$c, d, \text{err} (c)$	$c \xrightarrow{m_B} d, d \xrightarrow{j} c, d \xrightarrow{m_A} \text{err}$

k, c, d mean old-safe, inspection-clear and inspection-pending; c_0, c_1 are distinct controller states. Component/controller alphabets are exactly their displayed labels; unlisted edges are absent. Unlisted non-error monitor edges self-loop, and err absorbs every label. Each comma-separated label denotes an edge.

Endpoint tuples list stations A, B , controller and monitor. States h/e mean holding/empty; k, c, d mean old-safe, inspection-clear and inspection-pending; c_o, c_0, c_1 name controller states. Old reachable tuples are $x_A = (h, e, c_o, k)$ and $x_B = (e, h, c_o, k)$; new ones are $z_A = (h, e, c_0, c)$, $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Initial tuples are x_A, z_A and loadable targets are $\{z_A, z_B\}$, excluding pending inspection at y_B . The interval initializer records the holder at either all-old one-workpiece tuple. The new initializer admits all eight version-tagged one-workpiece tuples, installing d when B holds and c otherwise. The paths starting with A/B holding map to z_B/z_A at handover.

With empty-only transfers $g_A = g_B = \{(e, e)\}$ and exactly one workpiece, the two paths $\rho_B; m_B; \text{start}; \text{stop}; j; \rho_A$ from A holding and $\rho_A; m_A; \text{start}; \text{stop}; \rho_B$ from B holding have safe singleton outcomes: start installs inspection-pending if B holds and inspection-clear otherwise, stop removes the inspection ban, and both paths finish with new versions and clear inspection at matching supplied targets. Their remaining lengths $6, \dots, 0$ and $5, \dots, 0$ strictly decrease, proving finite completion from both reachable old states (13 policy states and 11 edges). Replacing the two transfer commands by one with relation $g_A \times g_B$, with endpoints and requirements fixed, requires both stations empty, but every ordinary move, inspection and requirement boundary preserves exactly one workpiece; that command can never execute, stays pending and makes every goal unreachable.

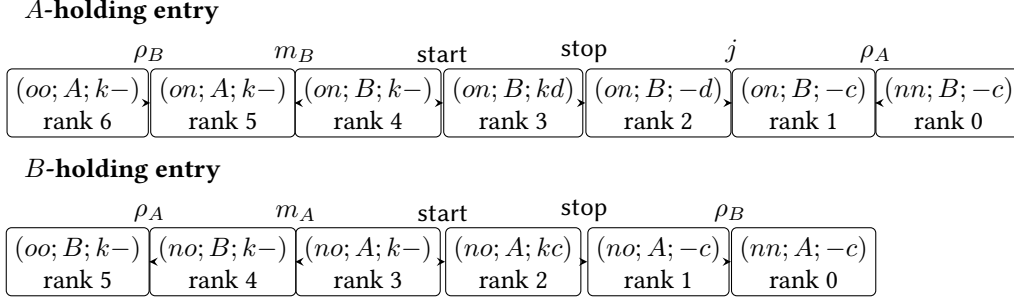


Figure A2: One policy from both entries. States list versions of A , B , the workpiece holder, and old/new monitors; the holder also determines the interval monitor. A dash means inactive. Every edge decreases the remaining-step rank. The two rank-zero states match the respective fixed-controller targets.

B.1 Residual Soundness and the Cell Reference

Residual soundness (RS) relates monitor memory to prior history. Let SafePref_r be requirement r 's prefix-closed language, and let $H_r(\xi)$ contain all admissible activation histories consistent with observed component state ξ . A new requirement's history includes start once; its initialized monitor reads only later events. Residual-sound initialization means

$$\underbrace{\{w : \eta_r^*(\iota_r(\xi), w) \notin F_r\}}_{\text{continuations accepted by the initialized monitor}} \subseteq \underbrace{\bigcap_{h \in H_r(\xi)} \{w : hw \in \text{SafePref}_r\}}_{\text{continuations safe after every admitted history}} \quad (\text{RS})$$

for every $\xi \in \text{dom } \iota_r$. Equality is exact; strict inclusion is conservative and may exclude feasible updates. An empty history set makes the right side universal. Synthesis checks initializer states and domains, but neither infers histories nor establishes their completeness.

For Cell's declared start-only reference, $H_{r_{\text{inspect}}}(\xi) = \{\text{start}\}$. Initializing at c is exact for that reference; initializing at d when B holds is conservative, forbidding departure until inspection. The inclusion proof follows below. This distinction exposes a separate obligation: an exact history residual need not encode the intended duty at activation.

A Boolean event-history variable, or *fluent*, records setting/resetting labels. For an initially true fluent p , a history making p false then true and the empty history end at the same value but differ under $\Box p$.

Deriving RS in the cell. The declared pre-activation observation retains only start, so $H_{r_{\text{inspect}}}(\xi) = \{\text{start}\}$ for every admissible cell tuple. The reference monitor starts at c , unchanged by start; the right-hand side of RS is therefore its non-error continuation language from c . If B is empty, initialization at c gives equality. If B holds, initialization at d is conservative: before the first j or m_B , d additionally forbids m_A ; either j or m_B brings the two initial states to the same state. Thus every word safe from d is safe from c , while the single event m_A witnesses strict inclusion. This is a proof for the declared observation scope, not recovery of pre-activation history.

Initializing at c while B holds satisfies equality but omits the intended pending-inspection duty encoded by d . Declared histories and intended obligations are therefore separate input choices.

C Policy Requirement-Boundary Argument

Opposing requirement boundaries. Policy(2) is our constructed example with two old/new requirement pairs: audit and access role; the number denotes the pair count. It requires audit overlap and role nonoverlap. Table A2 gives its complete input. Write o, n for old/new, u, l for unlogged/logged, A, R for audit/role, and ℓ_z, a_z for version- z log/access.

Table A2: Complete Policy(2) input; all events controllable. Unlisted plant/controller edges are absent; unlisted monitor edges self-loop, including at error err. Monitors have alphabet Σ .

Part	Finite definition
Plant/transfer	Each version has sole initial state r with loops on $\Sigma_N = \{\ell_o, \ell_n, \text{service}, a_o, a_n\}$. Transfer ρ changes its version with $g = \{(r, r)\}$.
Controllers	C^z ($z \in \{o, n\}$) has alphabet Σ_N , initial u , edges $u \xrightarrow{\ell_z} l \xrightarrow{\text{service}} u$, and a_z loops at u, l . An unreachable third state has all Σ_N loops.
Endpoint monitors	A_z : initial u , states u, l, err , edges $u \xrightarrow{\ell_z} l \xrightarrow{\text{service}} u$ and $u \xrightarrow{\text{service}} \text{err}$. R_z : initial ok, states ok, err, edge ok $\xrightarrow{a_z} \text{err}$, where $\bar{z}$ is the other version.
Interval monitor	Initial 1010; states $\{b \in \{0, 1\}^4 : (b_1 \vee b_2) \wedge \neg(b_3 \wedge b_4)\} \cup \{\text{err}\}$. Bits track active (A_o, A_n, R_o, R_n) . Respective stop/start commands write $b_1 = 0, b_2 = 1, b_3 = 0, b_4 = 1$; unsafe results enter err. Other events preserve bits.
Initialization	Four boundaries and ρ , once each; no precedence. Old monitors are A_o, R_o , new are A_n, R_n . On either (o, r) or (n, r) , initialize $A_n = u, R_n = \text{ok}$; initialize the interval to 1010 only at (o, r) . Domains are complete.
Entries/targets	For $x \in \{u, l\}$, entry q_x admits old tuple $((o, r), C^o = x, A_o = x, R_o = \text{ok})$; loadable targets are $z_x = ((n, r), C^n = x, A_n = x, R_n = \text{ok})$. Admission, pending commands, activation and goal tests follow Section 3.
Boundary merge	One global start activates both new monitors; one global stop deactivates both old monitors. Compose member observations audit then role; other inputs stay fixed.

From q_u , the saved policy executes $\text{start}_A, \text{stop}_A, \text{stop}_R, \rho, \text{start}_R$, reaching z_u . From q_l , service first reaches q_u . Ranks five/six decrease on every edge; other controllables are disabled. Figure A3 depicts this whole policy.

Both merged entries lose: the first global start overlaps roles; the first global stop removes audit coverage. Ordinary events and transfer preserve these bits, so every completing path encounters error. The saved checks confirm closure and both member orders. This compares boundary granularity, not DUCS expressiveness. Service continuity is not required. The interval monitor induces the opposite boundary orders without supplied precedence. Its role clause forbids the two role monitors from being active together; the access monitors alone only reject the other version's access event. The saved losing closure checks every alternative and both orders for observing merged member labels. Table A2 above gives the complete finite input. Figure A3 visualizes its saved policy and requirement lifetimes.

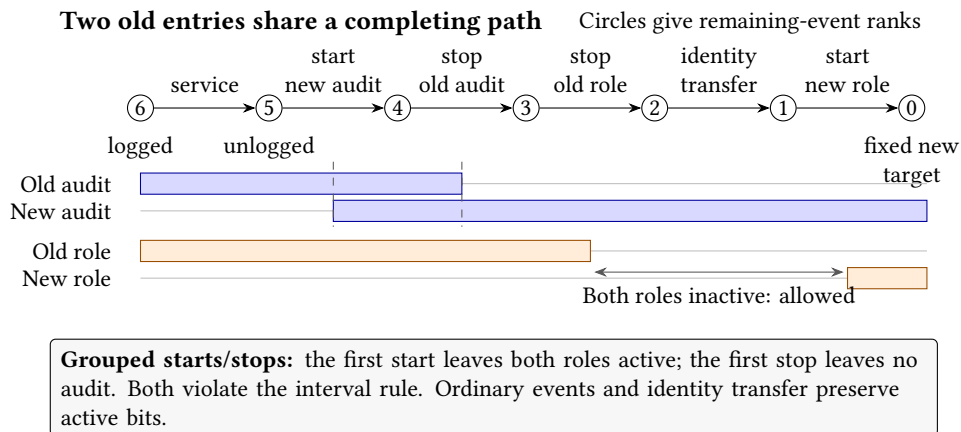


Figure A3: A saved Policy certificate with no supplied precedence; ranks need not be shortest. Logged/unlogged mark both entries; bars mark active requirements. New-audit start resets to unlogged; handover reaches new audit unlogged and new role valid. Arrows have one action and outcome. The audit overlap is required and the role gap is allowed; ordinary events may be withheld, with no service-continuity guarantee.

D Granularity and Safe Serialization

This section supplies the complete finite inputs and proofs for Section 5. The source-inspection results and auxiliary finite checks are reported with the evaluation records in Appendix P.

D.1 Finite contracts for the constructed families

These definitions specify the inputs behind Table 3 and the Rolling grid. They describe finite models, not implementations of deployment platforms. Component versions have distinct tags even when their transition graphs coincide. Unlisted plant and endpoint-controller edges are absent; unlisted monitor labels stutter, and errors are absorbing. Ordinary events are controllable unless explicitly called UC. There is no explicit command precedence. OLD/NEW monitors are empty except in Rolling+Audit; the stated interval monitors apply until handover. The fixed endpoint graphs determine every reachable old entry and the listed loadable new targets. The main paper defines quiescence, command execution and the compared merges.

Rolling(n, m) and block size k . For $2 \leq n \leq 6$, $1 \leq m < n$, each old component has one ready state with a $serve_i$ loop. Its new version has booting and ready states, with UC $ready_i$ from booting to ready and $serve_i$ only at ready. A local transfer maps old ready to new booting. The interval monitor starts at $r = n$, decrements r by the transferred block size, increments it on $ready_i$ (capped at n), and errors below m . Both fixed endpoints are all-ready tuples whose one-state controllers admit all $serve_i$ loops; these supply one entry and one target. Consecutive blocks of at most k , including the final remainder, are transferred atomically. The 15 (n, m) pairs compare $k = 1$ with $k = n$; the separate 70-cell grid varies $1 \leq k \leq n$. Proposition 5.2 proves the threshold for these inputs.

Canary(n, m). The same 15 (n, m) values use one old healthy state H and new states H, H_P, B_P, B . Only H serves. A transfer has two adversarial outcomes, H_P and B_P ; UC reports take H_P to H and B_P to B . A controllable restart takes B to H . The interval monitor is a bit vector initially zero: a broken report sets bit i , a restart clears it, and more than $n - m$ set bits cause error. Other labels stutter. This is a *reported-broken* budget; pending physical states are not identified with already reported failures. The saved certificate also checks physical health separately. The all-healthy old/new tuples and one-state serve-only endpoint controllers give one entry and one target. Merging transfers synchronizes their commands and retains every Cartesian-product outcome. The healthy-outcomes-only and no-recovery variants restrict transfer images or remove restart, respectively; their unchanged both-WIN and both-LOSS results delimit the recovery mechanism.

Canary certificate scope. The existing separate check passed on every state of all 30 returned WIN certificates: the 15 parameter pairs under Lazy and Direct-Full. It counts replicas in H or H_P against the threshold m ; B_P and B do not count. Thus it checks model-state health, while H_P still cannot serve. It does not check continuous service, every explored state or runtime behavior. The other 30 saved checks concern LOSS mechanisms, not physical-health success. Fine and merged transfers share capacity reasoning with Rolling, but their reported-health contracts are different; no safety-preserving reduction to the Rolling game is claimed.

DB-Rolling($3, m$). Here $m \in \{2, 3\}$. Each old component has primary P , secondary S , and election-pending E states; the new version adds booting B . Serving loops exist at P, S, E . A controllable $stepdown_{i,j}$ synchronously changes P_i to S_i and S_j to E_j ; UC $elect_j$ changes E_j to P_j . A transfer maps only S to B , and UC $ready_i$ changes B to S . The interval budget starts at three, decreases by the transferred block size, increases by one on readiness (capped at three), and errors below m ; election does not consume this nonbooting budget. The old endpoint is (P, S, S) , the new target (S, P, S) , both with serve-only controllers; unreachable padding retains the remaining ordinary alphabet. Thus the contract specifies a change of primary. With $m = 2$, a fine policy can change the primary and transfer

secondary components successively. The merged command requires all-secondary states, which cannot be quiescent while an election remains pending. With $m = 3$, even the first fine transfer violates the budget. These models do not assert uninterrupted write service.

Rolling+Audit. This is Rolling(2, 1) with controllable log_{old} and log_{new} loops in every local plant state. Each version's audit monitor has unlogged, logged and error states. Its own log sets logged; either serve changes logged to unlogged but errors from unlogged. The other log and all remaining labels stutter. The new initializer selects unlogged on all nine tagged physical tuples. The interval state is (r, o, a) , initially $(2, 1, 0)$: transfers/readiness change r as above, old-stop clears o , and new-start sets a . Safety requires $r \geq 1$ and $o \vee a$, enforcing overlap of the requirements. Each fixed all-ready endpoint controller alternates its own log with either serve; both its unlogged and logged states are entries/targets, and unreachable controller padding retains the remaining ordinary alphabet. These specify the memories, initializer and coverage obligation used in the comparison. Merging the two transfers loses; the single old-stop and new-start cannot be combined further, so merging requirement groups alone is only a renaming.

Threads(n, B). For $2 \leq n \leq 6$ and queue capacity $B \in \{1, 2\}$, each worker is idle or busy; component 1 additionally stores queue length $0 \leq q \leq B$. UC arrival increments q below capacity. At capacity, backpressure disables arrival whereas saturated arrival is a self-loop. Controllable $dispatch_i$ requires $q > 0$ and worker i idle, decrements q and makes that worker busy; for $i > 1$ it synchronizes with component 1. UC $finish_i$ returns a busy worker to idle. Old/new plant graphs coincide. Each local transfer is identity on idle states and preserves component 1's queue. There are no monitors. One-state endpoint controllers admit all ordinary labels; their reachable products from the empty, all-idle initial tuple give all old entries and all new targets. These two arrival modes yield 20 pairs. Under backpressure, disabling dispatch lets the finite queue fill and busy workers finish, permitting globally quiescent transfer. Saturated arrival can persist forever and prevents even fine transfer under the contract's quiescence rule. The negative control therefore distinguishes a genuine transfer opportunity from an implicit fairness assumption; it is not a reproduction of a multithreaded runtime.

D.2 Granularity need not preserve feasibility

The comparisons above do not assert that finer commands always retain every merged solution. The following analytical contract gives the reverse separation; it is not an additional measured instance or part of the 55-pair population.

Both components have one state per version and the shared ordinary alphabet $\{x, y, tick\}$, with x, y UC and $tick$ controllable. Every version has a $tick$ self-loop. The only other loops are: y in old component 1, x in new component 1, x in old component 2, and y in new component 2. All other transitions are absent. Shared labels synchronize, so neither x nor y is enabled in the all-old or all-new product; x loops in new/old and y loops in old/new. Each local transfer maps its unique old state to its unique new state. There are no requirements, initializers or precedence edges. The one-state old and new controllers loop on all ordinary labels. Their closed loops each have one reachable state and a $tick$ loop, hence are safe and deadlock-free; these states supply the entry and target.

With separate transfers, either first transfer reaches a mixed state with an enabled UC self-loop. The remaining transfer is then permanently blocked by the quiescence rule; the environment can repeat the loop forever, so every fine policy loses. Merging both transfers into their single Cartesian-product command instead moves directly from the quiescent entry to the quiescent all-new target, with rank one. Thus fine LOSS/merged WIN is possible. Atomic merging can remove a harmful mixed state, whereas Cell and Rolling show that it can remove a necessary intermediate state. The reported obstructions establish when a specified merge loses, not a universal refinement order between granularities.

D.3 A local condition for serializing transfer batches

The reverse separation does not imply that each coarse winner needs a joint implementation. The following condition uses local alphabets, enabled UC labels and transfer pairs. It concerns transfer-only merging: every requirement start and stop remains a singleton. No runtime atomic capability or reduction in synthesis cost is inferred from the condition.

For component i , let $A_i^v = \Sigma_{e_i^v} \cap \Sigma_{uc}$ and define its enabled and blocked UC labels at state s by

$$E_i^v(s) = \{a \in A_i^v \mid \delta_i^v(s, a) \neq \emptyset\}, \quad B_i^v(s) = A_i^v \setminus E_i^v(s).$$

An absent label differs from a blocked label: an absent component does not participate, whereas a blocked participant prevents synchronization. Call a transfer *UC-nonenabling* when every $(s, s') \in g_i$ satisfies

$$E_i^{\text{new}}(s') \subseteq E_i^{\text{old}}(s), \quad B_i^{\text{old}}(s) \subseteq B_i^{\text{new}}(s'). \quad (1)$$

Thus it offers no new UC label and retains every local UC blocker. The condition allows alphabet changes: an enabled label may disappear, and a blocked label may be added. With equal old/new UC alphabets, both inclusions reduce to enabledness inclusion. Monitors are passive and do not affect enabledness.

Proposition D.1 (Context-independent UC criterion). *Fix one local transfer pair (s, s') . Equation (1) holds iff replacing this local state preserves UC quiescence in every finite synchronous context whose other component states stay fixed. It also holds iff no such context can gain a globally enabled UC label through that replacement.*

Proof. Suppose both inclusions hold and a UC label a is enabled after replacement. All other participating components enable a . If the new component participates, $a \in E_i^{\text{new}}(s')$, hence $a \in E_i^{\text{old}}(s)$: the old component also participates and enables it. If the new component is absent from a , the blocker inclusion implies $a \notin B_i^{\text{old}}(s)$; the old component therefore either enables a or is also absent. In this case some other component participates, because a product event requires at least one participant. In both cases a was globally enabled before replacement. This proves nonincrease of the global UC set and therefore preservation of quiescence.

For necessity, the finite ordinary UC alphabet lets a one-state context block every UC label other than a chosen a : include those labels in its alphabet but give them no outgoing transitions. If $a \in E_i^{\text{new}}(s') \setminus E_i^{\text{old}}(s)$, omit a from the context alphabet. Initially a is absent or blocked in the component, so the product is quiescent. After replacement, a is enabled. If instead $a \in B_i^{\text{old}}(s) \setminus B_i^{\text{new}}(s')$, and the first inclusion has not already failed, then a must be absent from the new component. Give the context an a self-loop. The old component blocks it, but the new one no longer participates; again a quiescent product gains a . These witnesses establish both converses. $\square$

For example, removing a locally enabled UC label satisfies the criterion if all other local UC statuses stay unchanged, although alphabet equality fails. Removing a blocked label does not: another component may offer it. The quantifier over all contexts is essential. Failing the criterion need not enable UC in the supplied global contract, and need not preclude a fine winner. The result characterizes local preservation of quiescence; it does not characterize every safely serializable contract.

Restatement of Proposition 5.4. Let a fine contract and a transfer-only merged contract satisfy the comparison rules of Section 5: entries, endpoints, targets, initializers and ordinary behavior are unchanged; transfer outcomes are Cartesian products; active-monitor member effects commute on all monitor states; mapped precedence is irreflexive and acyclic. If every transfer is UC-nonenabling, a winning policy for the merged contract implies a winning policy for the fine contract covering the same old entries. A finite-memory serial expansion preserves the merged policy's selected endpoint continuations.

Proof. At a checkpoint, associate the two representations by equal component and monitor states, with every pending coarse block corresponding to all of its pending fine members. Consider a block B enabled by a coarse winner at checkpoint q . The state is UC-quiescent. Execute its members in any fixed order, disabling ordinary controllables between members.

No such member enables an ordinary UC event, by Proposition D.1, with the other components as its synchronous context. Induction gives quiescence at every partial prefix, so the next fine command is not blocked by UC.

Each remaining member retains its old source coordinate and its transfer image. The coarse block's enabledness makes each of these images nonempty. Valid mapped precedence has no edge within a block; all external predecessors required by that block have already completed. Thus every next member remains enabled. All of its outcomes are retained. After the last member, the set of component outcomes is exactly the coarse Cartesian product, and the pending commands correspond again.

The set of active monitors is constant throughout this transfer sequence. For each monitor, pairwise commutation makes the composite effect of the fine member order equal to the coarse block effect. Every partial outcome extends to a full product outcome. If an active monitor entered error in a partial prefix, its absorbing error would persist to that completed outcome, contradicting safety of every coarse outcome. Therefore every partial state is safe, and completed states have exactly the coarse successor's monitor values as well as its component coordinates.

Follow the coarse policy between checkpoints. Ordinary events and singleton boundaries have identical outcomes in the two representations. At a fine expansion of a block, the preceding argument covers every environment-chosen transfer outcome and excludes intervening UC events. This constructs a finite-memory fine policy: it stores the coarse checkpoint and the unexecuted members of the selected block. A coarse ranking bounds the number of checkpoint transitions; every expansion has at most $\max(1, \max_B |B|)$ events. Hence all its maximal runs safely reach a corresponding coarse goal in finitely many events. Goals have the same component and new-monitor coordinates and the same loadable endpoint states, so the same endpoint choice is valid and its continuation is unchanged. Memoryless sufficiency for the finite update game (Technical Appendix A) gives a memoryless fine winner, but does not require that winner to retain the serial expansion's endpoint choices. $\square$

Exact event overhead of the serial expansion. Let $n = |I|$ and let b be the number of blocks in the transfer partition, including singleton blocks. Contract entries have every transfer pending; a goal has none. Every completed merged run therefore executes each block exactly once. Its serial expansion replaces the single event for block B by exactly $|B|$ consecutive member events. It inserts no ordinary event or requirement boundary between members, and leaves all other events unchanged. Every expanded run projects to a merged run with the same completed block outcomes and selected endpoint, by the proof above. Consequently, a merged run of length ℓ expands to length

$$\ell + \sum_B (|B| - 1) = \ell + n - b.$$

If the merged policy's entry rank is r , its runs have length at most r , giving a bound of $r + n - b$ for the constructed finite-memory serial policy. The count stops at the handover-ready goal; it excludes handover installation and post-handover behavior, as does the original rank. This is an exact event overhead for that expansion, not an equality between optimal fine and merged ranks, a bound for every fine winner, or a time/downtime guarantee.

What the condition does and does not cover. The condition provides a local way to justify replacing an available atomic transfer batch by individual commands. Threads gives an application: its old/new graphs coincide and its idle transfers preserve all local states, including the queue, so both enabled and blocked sets are unchanged. Its finite-arrival winners can be serialized; the permanent-arrival losing controls remain losing. It is not necessary for a fine winner: Rolling introduces UC readiness events and violates the condition, but its capacity argument separately supplies a winning

sequence. The transfer condition alone does not justify splitting requirement-start or requirement-stop blocks: those splits change which monitors observe later member labels. Section D.4 gives the additional conditions. Safe adaptation sequences and individual/simultaneous reconfiguration are established topics [5, 36]; the proposition supplies a sufficient condition for this contract’s all-outcome finite-completion and fixed-endpoint requirements.

Finite UC phases alone are insufficient. A variant of the preceding reverse example removes UC cycles while retaining fine LOSS/merged WIN. Give both versions of each component states 0, 1, shared alphabet $\{x, y, tick\}$ and a controllable *tick* loop at each state. The only UC edges are $x : 0 \rightarrow 1$ in new component 1 and old component 2, and $y : 0 \rightarrow 1$ in old component 1 and new component 2. Every transfer maps old j to new j for $j \in \{0, 1\}$, so the relations are total. Both fixed endpoint controllers have one state and loops on all three labels. From their initial $(0, 0)$ tuples, no UC label synchronizes, making $(0, 0)$ the only reachable tuple and the only loadable target. After either first fine transfer, one UC event is forced and changes both coordinates to 1. UC then terminates, but the remaining transfer can only reach the incompatible new tuple $(1, 1)$; no ordinary event restores 0. The joint transfer reaches new $(0, 0)$ directly. These are analytical examples, outside all measured populations. They show why finite UC activity and total transfer relations alone do not establish safe serialization; matching the fixed endpoint also matters.

D.4 Serializing requirement-boundary batches

Transfer and boundary batches have different intermediate-state risks. A boundary leaves components unchanged, but splitting it makes a newly started monitor observe later starts, or a monitor awaiting removal observe earlier stops. The following sufficient conditions account for precisely these additional observations.

Let $\mathcal{B}$ partition all update commands into type-homogeneous blocks, as in Section 5. For a start block, write B for its new requirements; for a stop block, write B for its old requirements. Impose:

1. Every component in a transfer block of size greater than one is UC-nonenabling as defined in Section D.3.
2. For each start block B , distinct $r, s \in B$, and tuple $\xi \in \bigcap_{u \in B} \text{dom } \iota_u$,

$$\eta_r(\iota_r(\xi), \text{start}(s)) = \iota_r(\xi).$$

3. For each stop block B , distinct $r, s \in B$, and $m \in M_r \setminus F_r$,

$$\eta_r(m, \text{stop}(s)) \notin F_r.$$

The conditions on singleton blocks are vacuous. The start condition concerns initialized states, whereas the stop condition concerns every safe state but need not preserve the state itself: that monitor will be removed before the checkpoint. The existing comparison rules still require commuting member effects on every monitor that remains active throughout a block, valid mapped precedence, and unchanged entries, endpoints, initializers, targets and ordinary behavior.

Combined serialization result. Under these conditions, a merged winner has a finite-memory serial expansion that wins the fine contract from the same entries and preserves its selected endpoint continuations. Each completed run gains exactly $|\Sigma_{\text{upd}}| - |\mathcal{B}|$ events. Thus a merged entry rank r gives the serial policy a bound of $r + |\Sigma_{\text{upd}}| - |\mathcal{B}|$ events to a handover-ready goal.

Proof. Use the checkpoint construction of Proposition 5.4. Singleton blocks already agree. A nonsingleton transfer block is covered by its proof; the local transfer condition is needed only for such blocks.

For a boundary block, fix any member order and disable other controllable events until the block completes. No member changes ξ , so the coarse command's UC-quiescence persists throughout. Valid mapped precedence introduces no internal dependency, and all external predecessors have completed. In a start block every remaining initializer stays defined at the unchanged tuple. Newly initialized monitor t_r begins at $\iota_r(\xi)$; each subsequent member start leaves it there by condition 2. It is therefore safe at every prefix and matches the coarse initialization afterward. In a stop block, each not-yet-removed member starts safe and remains safe under earlier stops by condition 3. Its own stop removes it without observation, exactly as in the coarse block; its intermediate state need not match a coarse state.

Monitors outside the boundary block that are active at the checkpoint remain active. Their commuting member effects give the coarse final value. An error in a proper prefix would persist by absorption to that final value, contradicting safety of the coarse successor. Monitors inactive throughout stay inactive. Hence each boundary sequence is enabled and safe, with exactly the coarse successor's component, monitor and pending-command coordinates at completion.

The checkpoint argument therefore applies to every block. Every serial run projects to a merged run with the same completed outcomes and selected endpoint. Each command is pending at admission and executed once before a goal, so every completed merged run uses each block once. Replacing block B by its $|B|$ members adds $\sum_{B \in \mathcal{B}} (|B| - 1) = |\Sigma_{\text{upd}}| - |\mathcal{B}|$ events and no others. The merged rank bounds the projected run, giving finite completion and the stated event bound. As before, memoryless sufficiency yields a fine winner, while endpoint-selection preservation and the exact overhead are claims about the constructed serial policy. $\square$

Why the extra boundary conditions matter. Omitting either boundary condition admits a reverse separation. For both examples, take one component with one state per version, a controllable *tick* loop, and an identity transfer. Both endpoint controllers have one state and admit *tick*. There are no interval requirements or precedence edges. Every monitor has safe state 0 and absorbing error 1; unspecified labels stutter.

For starts, use no old requirements and two new requirements r, s , each initialized to 0 on every tuple. From 0, t_r enters error on $\text{start}(s)$ and t_s on $\text{start}(r)$. The all-new target has both monitors at 0. Joint start installs both safely; any split order makes the first monitor error on the second start. For stops, instead use two old requirements with initial state 0 and no new requirements. Monitor t_r errors on $\text{stop}(s)$, and t_s on $\text{stop}(r)$. Joint stop removes both safely; either first individual stop errors the other monitor before its removal. The new target is the sole new component/controller state.

Each merged contract wins in two events (one transfer and one boundary batch), while every fine policy loses. Endpoints are safe and deadlock-free, transfer is UC-nonenabling, and member effects commute: for each monitor they consist of an identity and an error-setting map. Thus these examples isolate the additional boundary observations. They show that the conditions cannot simply be dropped, not that they are necessary for every serializable contract. Both are analytical contracts outside the measured populations.

This result gives a sufficient coarse-to-fine implication, not equivalence or a guarantee for arbitrary scheduling between members. The measured boundary comparisons are unchanged; no new synthesis is used to establish the result. A source declaration that omits boundary labels can support these conditions, but applying them to a compiled game additionally requires the corresponding monitor transitions and comparison rules. We do not count unexported OLD testers as checked automata. The event bound excludes handover installation and post-handover behavior and is not an elapsed-time bound.

E NEW and UPD History-Scope Derivations

This appendix gathers the complete criteria and proofs for prior-history and entry-scoped requirements. S3 gives the measured frontend and its boundary conventions. These scopes are distinct from

activation-scoped invariants in Appendix F; none establishes platform conformance.

E.1 Activation histories and residual meanings

For prefix-closed requirement language SafePref_r , let $H_r(\xi)$ be the declared activation histories projected by obs_r . NEW checks use safe prior histories (A); UPD checks begin their reference at entry (E). Whole-history meaning H is a separate diagnostic.

NEW activation histories include start once: initialization supplies the post-start residual and online monitoring consumes later events. Game admission initializes monitors without an update-game event; the FSP boundary interpretation below accounts for `hotSwapIn` once in the supplied UPD boundary state. Define $\mathcal{C}_r(\xi) = \bigcap_{h \in H_r(\xi)} \{w : hw \in \text{SafePref}_r\}$ and $\mathcal{L}_r(m) = \{w : \eta_r^*(m, w) \notin F_r\}$. Residual-sound initialization requires

$$\mathcal{L}_r(\iota_r(\xi)) \subseteq \mathcal{C}_r(\xi) \quad (\xi \in \text{dom } \iota_r) \quad (\text{RS})$$

with equality for an exact residual and strict inclusion for a conservative one. Distinct activation histories may require finite history bits in components and transfers. RS is supplied external interpretation evidence relative to the given requirement and observation meanings. For $r \in R^{\text{upd}}$, $\text{dom } \iota_r$ includes every reachable old component state.

E.2 Safe prior histories for NEW requirements (A)

A new requirement becomes effective at its own activation. Its supplied meaning retains the progress or obligations represented by the current fluent vector, while excluding prior violating histories from the reference intersection. With at most one occurrence of each update event, define

$$H_r^{\text{safe}}(\xi) = \{h \in H_r(\xi) : \eta_r^*(m_r^0, h) \notin F_r\}, \quad \mathcal{C}_r^{\text{safe}}(\xi) = \bigcap_{h \in H_r^{\text{safe}}(\xi)} \mathcal{L}_r(\eta_r^*(m_r^0, h)).$$

Then RS_A is $\mathcal{L}_r(\iota_r(\xi)) \subseteq \mathcal{C}_r^{\text{safe}}(\xi)$. This is a choice of requirement meaning, not a proof that a past violating trace satisfied the original invariant. In particular, conditioning on safe histories does not validate application intent. If $H_r^{\text{safe}}(\xi)$ is empty, the intersection is the universal continuation language: inclusion is immediate, but equality generally fails. Exactness below is therefore explicitly restricted to nonempty safe-history domains.

Lemma E.1 (A: language-determined safe-prefix initialization). *Explore the freely reachable monitor/observer product over one-shot update histories, recording but not expanding ERROR pairs. Suppose all non-error monitor states reached at the same fluent vector have equal continuation languages. On the activation domain, let the actual non-error lookup $\sigma_r(v)$ have that common language. Then $\iota_r(\xi) = \sigma_r(f(\xi))$ satisfies RS_A , with equality whenever $H_r^{\text{safe}}(\xi) \neq \emptyset$.*

Proof. Every admissible safe history reaches a non-error product pair with observer vector $f(\xi)$. By language equivalence and lookup compatibility, its residual language equals $\mathcal{L}_r(\iota_r(\xi))$. A nonempty intersection of these equal languages is that language. For an empty history set, the universal intersection contains the initialized language, proving the stated inclusion. $\square$

Language equivalence, rather than equality of state identifiers, is also the frontend's acceptance condition for multiple non-error states at a lookup vector. Our independent checker compares deterministic state pairs until an ERROR/non-error distinction or pair closure, and separately reconstructs the frontend's error-priority, equivalent-state/minimum-identifier rule. This checks the earlier saved monitor/observer exports against the source rule; those exports do not contain a materialized lookup table. A separate fresh frontend export directly checks all 138 tables (S4), while leaving general physical-state decoding and actual activation histories unverified. Appendix F.3 separately analyzes two saved Railcab exports. The check establishes RS_A inclusion for all 138/138 NEW initializers without a nonemptiness premise. Equality additionally requires nonempty safe histories at the actual activation; their plant-level nonemptiness remains unverified.

E.3 From update entry: UPD requirements (E)

An update-interval requirement becomes effective at entry. Its reference monitor starts afresh with the actual entry fluent vector v , consumes `hotSwapIn` once, and observes the subsequent update interval. Let $\mathcal{L}_r^E(v)$ denote the resulting continuation language and b_r the frontend's constant post-entry monitor state. For the entry vectors $V_r(\xi)$ compatible with ξ , define

$$\mathcal{C}_r^E(\xi) = \bigcap_{v \in V_r(\xi)} \mathcal{L}_r^E(v), \quad \text{RS}_E : \quad \mathcal{L}_r(b_r) \subseteq \mathcal{C}_r^E(\xi).$$

The reference language must account for the effect of v on monitor initialization; simply attaching an unused observer vector to the fixed compiled DFA does not establish that relation. Before and after the one entry occurrence, each other update label is permitted at most once in the sufficient-condition overapproximation. This overapproximation may admit update labels before entry that the actual game excludes.

Lemma E.2 (E: entry-value-independent boundary language). *Let V_r overapproximate all reachable entry observer vectors under the one-shot event constraint. If $\mathcal{L}_r^E(v) = \mathcal{L}_r(b_r)$ for every $v \in V_r$, then the constant initializer satisfies RS_E exactly at every actual entry with nonempty $V_r(\xi) \subseteq V_r$.*

Proof. Every actual entry vector belongs to V_r , so its reference post-entry language equals the boundary language by hypothesis. Thus the nonempty intersection over $V_r(\xi)$ consists of copies of $\mathcal{L}_r(b_r)$ and equals that language. Constant initialization uses b_r , giving equality in RS_E . $\square$

Lemma E.3 (E': update-stage fluents suffice). *Suppose an UPD monitor references only declared fluents whose initiating and terminating labels are update commands: `stopOldSpec`, `startNewSpec`, `reconfigure` (including their indexed forms), or `hotSwapIn`. No such label occurs before actual entry. If the constant initializer is the non-error boundary state b_r obtained from compiled state zero by consuming `hotSwapIn` once, then it satisfies RS_E with equality at every actual entry.*

Proof. Before entry, no defining update label occurs, so all referenced fluents retain their declared initial values. The fresh E monitor therefore starts at state zero and consumes `hotSwapIn` once, reaching b_r . Every actual entry has reference language $\mathcal{L}_r(b_r)$, so the reference intersection equals that language. $\square$

The E' check reads the expanded initiating/terminating sets of each referenced fluent from the saved model exports, checks the constant initializer and its non-error boundary, and certifies 89/135 UPD occurrences. It uses the actual game's exclusion of pre-entry update commands; the older free-history diagnostic admits such commands and is retained unchanged. A name such as `stopPump` is an ordinary plant event, not an update command.

The historical E' condition leaves 46 R1 occurrences unverified because they reference ordinary-action predicates. Attaching different observer vectors to a fixed DFA does not establish their reference languages. The additional source-specific check below proves their entry interpretation without changing those historical records.

Lemma E.4 (E: guarded-invariant entry initialization). *For a fresh entry-scoped requirement $\Box(G \Rightarrow \neg A)$, suppose G is initially false, changes only on actual update commands excluded from old-endpoint histories, and is unchanged by entry. Let A be a disjunction of ordinary-event predicates, each reset by entry. If the saved DFA has the two-state invariant transitions described below and its constant initializer is the non-error false-guard state obtained by consuming entry once from its default initial state, then the initializer satisfies RS_E with equality at every actual entry.*

Proof. No update command occurs before entry, so G is false there. Every possible value of the action predicates satisfies the initial implication. Entry leaves G false and resets all action predicates; the fresh pure invariant has no pending obligation from an earlier position. The resulting continuation

language is thus identical for every entry valuation and equals that of the supplied boundary state. The compatible entry set is nonempty at an actual modeled entry, so intersecting these equal languages establishes exactness. $\square$

The added checker reads the original source formulas, expands action aliases, and checks their correspondence to saved monitor/observer exports. For each R1 occurrence, it requires one default-false declared guard, nonempty disjoint initiating/terminating sets consisting only of actual update commands, and ordinary-event predicates reset by entry. It checks source/export equality of these sets, a constant non-error boundary initializer, and all DFA transitions against the two-state invariant: initiating commands set the guard, terminating commands clear it, a forbidden ordinary event when the guard holds enters ERROR, and other events stutter. ERROR absorbs all labels; absent local-alphabet edges become ERROR and labels outside that alphabet stutter, matching the adapter. All 46 monitors omit the entry label from their local alphabets, so consuming entry stutters at the false-guard state.

Inspection of namespace validation and endpoint construction, together with the saved successful input-validation results, establishes that old histories contain only ordinary events. The adapter computes the boundary value once and installs it directly at every old root; initialization does not consume entry a second time. Seven negative mutations test rejection of a wrong guard value, non-update guard event, guard-changing entry, missing declared edge/label, non-rejecting forbidden event and missing entry reset. The checker verifies all 46 occurrences across 4,722 state-label pairs and 854 possible action-predicate entry valuations. These are static validation counts, not extra synthesis trials or enumerated physical histories. S4 preserves the older free-history and E' results, and the new CSV labels its distinct sufficient condition. The new proof uses the existing E meaning; it does not establish the history-inclusive H interpretation or application-level conformance.

E.4 Whole-history obligations: a stronger diagnostic

For this coverage audit, obs_r retains the events defining the referenced fluents, independently of the active component version. Run the requirement monitor from m_r^0 on the whole projected history before activation and its continuation. With absorbing error states this gives

$$\mathcal{C}_r(\xi) = \bigcap_{h \in H_r(\xi)} \mathcal{L}_r(\eta_r^*(m_r^0, h)).$$

H counts violations before activation. If an admitted history reaches ERROR, its continuation language is empty; a non-error initializer accepts the empty suffix and cannot satisfy RS at that activation. We retain the H check as a diagnostic and assert no benchmark RS guarantee under H. The A and E interpretations above have different scopes; none changes the compiled game or any saved synthesis decision.

Lemma E.5 (Fluent-determined exact initialization). *Let the freely reachable product of the deterministic requirement monitor (including absorbing errors) and all its referenced fluent observers start at their specified initial states. Suppose equal fluent vectors in that product imply equal monitor states. Let $\sigma_r(v)$ return that unique monitor state for each reachable vector v . For any activation state with $H_r(\xi) \neq \emptyset$ whose histories are represented in this product and whose supplied initializer is $\iota_r(\xi) = \sigma_r(f(\xi))$, initialization is an exact residual for the history-inclusive interpretation.*

Proof. Each projected activation history reaches a product pair $(\eta_r^*(m_r^0, h), f(\xi))$. Fluent determination makes its first coordinate equal to $\sigma_r(f(\xi))$, independently of h . Thus every language in the intersection defining $\mathcal{C}_r(\xi)$ is the same language $\mathcal{L}_r(\iota_r(\xi))$. Because the history family is nonempty, its intersection is that language, proving equality in RS. For an empty history family, the intersection is Σ^* and this argument gives inclusion only, not equality in general. $\square$

A single non-error state is insufficient for this lemma. For $\Box p$ with p initially true, let a make p false and b restore it. The empty history and ab have the same current fluent value but reach the safe

and absorbing-error monitor states, respectively. A check that discards errors verifies only safe-prefix determination, a weaker condition that needs a separate safe-history premise. We use the full product, including errors. Failure of this plant-independent sufficient condition leaves RS unverified for the actual activation histories; it is not by itself a counterexample to the supplied plant's RS.

F Activation-Scoped Initialization

Obligations that begin at activation. Activation-scoped interpretation ACT starts a pure invariant with the actual post-boundary observer vector. It retains factual fluent values, without a past-violation bit, and consumes start once. The following proposition gives one complete criterion: a matching safe reference residual is exact even when the actual history violated the invariant before activation. This differs from A's safe-history restriction and H's whole-past obligation. Section 6.2.1 checks ACT for the thirteen invariants along the saved PC2 policy; the 138 A checks remain A checks.

F.1 From a Safe Lookup Cell to an Activation-Scoped Invariant

The following proposition gives the complete statement and proof for the lookup criterion in main-paper Section 4.2. A faithful invariant monitor reached safely with observer vector v accepts exactly $\text{ACT}_\varphi(v)$. The witness may come from the unrestricted monitor–observer product; physical executability of that reference history is unnecessary. We first check the constructed lookup, then compare its monitors with source formulas, and finally identify the conditions required at actual activations. Main-paper Table 2 maps the guarantees; Table A9 in Appendix P.1 gives detailed validation populations.

An invariant that begins at activation must hold immediately after start and after every later event. We call this the activation-scoped interpretation, written ACT. For a pure invariant $\Box\varphi$, its truth depends on the current values of event-tracked properties (fluents). Deterministic observers update these values through $d : V \times \Sigma \rightarrow V$. From the actual post-start vector v , the permitted continuations are

$$\text{ACT}_\varphi(v) = \{w \mid \forall u \preceq w : \varphi(d^*(v, u))\},$$

where prefixes include the empty word. A faithful safety monitor M has non-error language $S_\varphi = \text{ACT}_\varphi(v_0)$ from its initial state. Let $L_M(m)$ denote its continuations from state m .

Proposition F.1 (Safe lookup cells for pure invariants). *If a lookup at v selects a non-error state m reached together with v by a safe word in the unrestricted monitor–observer product, or a state with the same continuation language, then $\varphi(v)$ holds and $L_M(m) = \text{ACT}_\varphi(v)$.*

Proof. Let h reach the pair (m_h, v) . Faithfulness and non-error reachability give $h \in S_\varphi$, hence $\varphi(v)$. For every suffix w ,

$$w \in L_M(m_h) \iff hw \in S_\varphi \iff w \in \text{ACT}_\varphi(v).$$

The second equivalence uses safety of the prefix h and deterministic observers: future valuations depend only on v and w . Language equivalence transfers the equality to m . $\square$

Reconstructing the frontend lookup. The nine inherited inputs have 46 NEW requirement positions and three variants, yielding 138 occurrences. The saved independent reconstruction checks all 138 lookup tables and 1,587 entries, including 900 non-error entries, 687 error entries, 408 observer columns and 31,176 observer transitions. S4.2 gives the export protocol and reconstruction checks. A non-error entry is chosen from a pair reached by a finite safe word of the free monitor–observer product; equivalent-state alternatives must have the same error language. The reconstruction explores non-error successors, so induction on its search depth supplies that witness. The saved table records (v, m) , not the witness word itself.

Matching monitors to source invariants. The reconstruction establishes safe lookup reachability. To check the proposition’s separate faithfulness premise, we compare the saved monitors with the original source invariants independently of the Java formula compiler. All 46 NEW positions are pure invariants $\Box\varphi$: seven use declared fluents only, six use action predicates only, and 33 use both. Names such as Pending and SEQUENTIAL designate deterministic current-value observers, not additional temporal operators. A restricted parser reads propositional negation, conjunction, disjunction and implication inside $\Box$, assertion aliases, fluent initial values, event-set aliases/differences and finite indexed ranges. Unsupported syntax is rejected. Source-defined observer rules must match the saved rules, including action predicates’ last-event convention.

For each invariant, the finite check verifies $\varphi(v_0)$ and a non-error initial monitor state, then explores every non-error monitor–observer pair. At each pair (m, v) and every checked label a , it verifies

$$\delta_M(m, a) \in \text{Error} \iff \neg\varphi(d(v, a)).$$

The checked alphabet contains every saved observer label and every source-referenced event, plus a fresh representative for all unmentioned events. The latter class has the same source-observer effect (declared fluents stutter and action predicates reset); the saved DFA uses its default transition. From the initial pair, induction on word length now proves that a prefix is non-error exactly when all its source valuations satisfy φ . Thus this finite condition establishes the monitor-faithfulness premise used above on these alphabets; it is not an assumption that a passing table check alone validates the compiler.

All 138 occurrences pass: 900 non-error product pairs, 35,283 pair/label checks, and all 900 non-error lookup cells contained in the safe product. There are no unsupported formulas or mismatches. The three variants agree on source formula, observers and monitor for each of the 46 positions; the counts are occurrences, not 138 independent formulas. This establishes ACT exactness for those lookup cells under their reference observation semantics.

What this yields for a winning policy. At a real activation, the installed state must have the ACT language for the actual post-start observer vector. Using that vector in the lookup with the same future observation rules is sufficient; Appendix F.2 gives a further sufficient case for command-preserved observers. Start is accounted for once, before initialization, rather than observed twice. Every active suffix of a winning policy is accepted by the installed monitor; the proposition therefore gives the invariant at activation and after every observed prefix. A previous actual violation is irrelevant to this explicitly start-scoped requirement. This is not the history-inclusive H interpretation, nor does it turn an arbitrary A inclusion into ACT exactness. An undefined/rejected lookup or a separate fallback initializer is outside this argument. Conversely, $\varphi(v)$ alone does not ensure that the table has a non-error entry for v .

Remaining activation conditions. Appendix F.2 establishes when preserving physical action observers across commands gives the same ACT language as resetting the reference predicates. Its conditions hold for all 46 inspected NEW positions and their 138 occurrences. Correct initial observer values, raw-vector decoding, column projection, ordinary-event updates and actual lookup use remain premises. Appendix F.3 checks the decoded keys and source observations in two saved Railcab exports, accepting their exported old-entry vectors. Neither analysis verifies historical lookup/fallback calls or every benchmark activation. These results do not report 900 observed activation points or validate all 25 WIN policies at source level.

The existing objects are `results/supplement-checks/frontend_lookup_demo/verification_details.json` (per-requirement normalized actual and independently reconstructed tables) and `lookup_check_summary.csv`; `verify_saved.py` contains the reconstruction and exact table comparison. The additional source analysis is supplied in `results/supplement-checks/source_invariant_lookup/`. These checks reuse saved objects and introduce no new synthesis or performance result.

F.2 Preserved Observers Across Update Commands

The physical observer component preserves its values across update commands. Under the source interpretation, declared fluents can stutter while an implicit action predicate, which records the last event, becomes false. Unequal vectors alone do not establish unequal requirement meanings: the relevant object is the continuation language at the selected lookup cell.

Lemma F.2 (Command stuttering at a safe lookup cell). *Let M be faithful for $\Box\varphi$ under deterministic observation rule d , as in Proposition F.1. Let c preserve every non-error state of M . If (m, v) is safely reachable in the unrestricted monitor–observer product, then*

$$L_M(m) = \text{ACT}_\varphi(v) = \text{ACT}_\varphi(d(v, c)).$$

The equality also holds for an initializer language-equivalent to m .

Proof. Let h be a safe witness reaching (m, v) . Since c preserves m , hc is safe and reaches $(m, d(v, c))$. Applying Proposition F.1 to the two witnesses gives both equalities. Language equivalence transfers them to the selected initializer. Repeating this argument handles a finite sequence of such commands. $\square$

Application to last-event predicates. Partition a requirement’s vector into declared fluent values f and implicit action predicates p . Suppose no update command initiates or terminates a declared fluent, and no action predicate denotes an update command. A command then has reference effect $R(f, p) = (f, 0)$, while the physical observer preserves (f, p) . An ordinary event updates both observers identically, and its action-predicate values depend only on that event; in particular, $d(R(v), a) = d(v, a)$ for an ordinary a . With initially matching vectors and correct ordinary-event updates, induction over the event sequence shows that the reference vector equals the physical vector after an ordinary event and its R -image after one or more commands. Because activation itself is a start command, the latter is the relevant case at a NEW activation.

If the raw physical vector v selects a non-error cell covered by Proposition F.1, Lemma F.2 therefore gives the exact ACT language for the reference post-start vector $R(v)$. Subsequent monitor transitions must still consume each modeled event once. The result permits this specific difference in observer storage; it does not infer correct decoding, column projection or lookup use from the table alone. Rejected or missing raw cells and fallback initializers require separate consideration.

Complete saved-input check. We identify the actual stop, start and transfer commands from each source’s declared command sets, the selected old/new safety goals, and every entry of its used transfer relations. These agree with the saved Base/R1/R2 contract declarations; static inspection of the experiment JAR confirms the command-construction rules. Across all nine inputs there are 112 commands, counted separately per input. None occurs in a NEW declared fluent’s initiating/terminating sets or denotes a NEW action predicate. All 138 saved NEW monitors exclude these commands from their alphabets and preserve every non-error state on them under the exported monitor decoder. Entry/finish markers are checked separately and are also outside those monitor alphabets. All three variants have the same monitor for each of the 46 source positions.

As an additional source-only check, we enumerate every Boolean assignment to each position’s atoms, including unreachable assignments. All 896 assignments satisfy $\varphi(v) \Rightarrow \varphi(R(v))$; there are no counterexamples. Thirty-nine positions contain action predicates; for the remaining seven, R is the identity. This truth-preservation check is weaker than residual-language equality; the latter follows from the safe-witness argument and monitor faithfulness above.

We also project all 1,587 saved lookup cells through R , decoding each observer’s Boolean values from its initial truth and complete saved transitions. All 900 non-error cells map to existing non-error cells with the same selected monitor state; 273 keys change. Of the 687 error cells, 357 map to non-error and 330 remain error. No projected key is missing. This is a transformation of table cells, with three occurrences per source position, not a count of observed or physically reachable activations.

A remaining restriction on activation. For GSM’s invariant $\Box \neg \text{decode}.2$, the action-predicate key 1 maps to error, while its reset key 0 selects safe monitor state 0. Thus a raw observation can be rejected even though its post-start reference value admits safe continuations. The 357 error-to-non-error projections expose this possible extra restriction; they do not establish that these keys occur at actual starts. Exact continuation languages at accepted cells do not imply that the initializer admits every ACT-valid activation. Synthesis remains complete for its supplied initializer domain. We retain that domain and all measured outcomes.

Four copied-input fault injections are detected: a fluent observing a command, a monitor entering error on a command, a positive event predicate broken by reset, and a changed lookup selection with a different continuation language. The last yields distinguishing suffix `checkCrossingStatus` at a Railcab cell. Original inputs remain unchanged.

The scripts reuse the archived restricted source parser, Boolean evaluator and monitor-language checker. They inspect saved sources and exports without new synthesis or performance measurements. Their inputs, complete outcomes, bytecode excerpts and negative controls accompany the manuscript source in `evidence/command_observation/`. These finite checks establish the stated command conditions for the inspected inputs. General initial-observation and activation decoding, ordinary-event conformance, actual lookup/fallback calls and runtime behavior remain outside the analysis; Appendix F.3 separately checks the two saved Railcab policies.

F.3 Source-Observer Analysis of the Saved Railcab Policies

This analysis uses the complete, previously saved Base and R1 policy exports from S4.4.3 and the independently exported frontend lookup tables. It performs no new synthesis or performance trial. Both exports cover 22 old entries; they share the same exported new-controller graph. We accept the observer values at the saved old entries as the starting values, rather than reconstructing their preceding old-system histories.

Decoding and provenance. The saved policy and lookup metadata identify the same input and experiment JAR. Static inspection of that JAR confirms that the adapter orders global observers by name, preserves that order in local-state arrays, and prints those arrays in the saved physical-state strings. All ten Railcab observers start false; the fluent-to-LTS conversion represents false by 0 and true by 1 in this case. The convention reverses for initially true fluents and is not assumed generally. The saved observer transitions also agree with this decoding. The analysis matches each observer to its source fluent or event predicate by its initial value and initiating/terminating sets, then projects the global vector into the actual per-requirement lookup column order.

Finite checks and why they cover the saved continuations. For each exported old entry, we traverse the saved policy with an independently updated source-observer vector. Each edge consumes its label once, including transfer, stop and start commands; handover connects to the selected post state without a further source event. At every reached pair, we compare the source vector with the decoded saved vector and evaluate each active NEW invariant directly from its source formula. At a start edge, we additionally project the post-start vector into the exported lookup table and compare its value with the newly installed monitor state.

The traversal closes over all saved outcomes and all transitions reachable from the selected post states. Induction on a path prefix shows that the traversed vector is exactly the source observer valuation from the accepted entry vector. A passing truth check at every reached active pair therefore establishes each invariant from its start along every saved continuation, including every finite prefix of an infinite post-handover execution. This direct truth argument does not infer source validity merely from the absence of a monitor error. The lookup comparison separately connects the reached initializations to the reference-language result of Proposition F.1.

All 21 start edges select the installed non-error state, all reached decoded vectors match the source replay, and all seven source invariants hold whenever active. All saved update states are reached. The

Table A3: Complete saved Railcab policy coverage. Each policy has 22 entries. The final column refers to the same shared post graph, not two independent applications.

Policy	Update states / edges	NEW start edges	Handover targets	Post states / edges
Base	115 / 118	14	2	70 / 116
R1	44 / 46	7	1	70 / 116

selected post closure contains 70 of the 71 exported post states. After the first NEW start, both saved update policies contain only further starts before handover: no ordinary update edge occurs in that suffix. The nontrivial ordinary continuation check is in the shared post graph. Counts of repeated requirement checks are not independent instances or applications.

A nontrivial initial state in the branching policy. Let a mean that approachingCrossing has occurred since endOfTS, let e mean that the last emergency-brake point has occurred since that event, and let c be the checkCrossingStatus event predicate. The source requirement is $\Box(c \Rightarrow (a \wedge \neg e))$. The upper branch of main-paper Figure 2 starts this requirement with key $(a, e, c) = (1, 0, 0)$ and monitor state 1; the lower branch starts at $(0, 0, 0)$ and state 0. R1 uses the former key. The saved monitor permits checkCrossingStatus from state 1 and rejects it from state 0. Thus always resetting to the initial state 0 would erase the already observed approach; equality of initial-state identifiers is not an adequate handover test.

Rechecking and boundaries. The accompanying manuscript source includes `evidence/railcab_saved_activation/`: the analysis script, an unchanged input subset, per-edge results and three negative controls. Run `analyze_saved_railcab.py` with `--artifact-root inputs --output` followed by a new result filename. The analysis reuses the restricted source parser and Boolean evaluator from the earlier source-invariant checks; it is not a second independent parser. The controls change an installed monitor state, a non-entry observer bit, and a post-graph event, respectively. Each is rejected, including the bit change whose only failure is observer inconsistency.

The checks rely on the exported policy edges, active-requirement masks and handover targets. They do not rederive the old-entry vectors, recover historical lookup calls or establish which fallback branch executed. They do not validate physical execution, unselected policies, other applications or a general FSP compiler. The separate command-observation argument in Appendix F.2 addresses safe lookup cells under its explicit observation premises; matching on these saved paths does not establish those premises for every policy.

A source-derived policy with active ordinary behavior. This is an analytical construction from the supplied `GSM_FG.lts` and saved frontend lookup, not an exported solver policy or an additional experiment. The old plant is the cycle

$$\begin{array}{l}
 \text{SENDER} \xrightarrow{\text{shift}X} X \xrightarrow{\text{shift}Y} Y \xrightarrow{\text{shift}Z} Z \xrightarrow{\text{encode}} \text{ENCODED} \\
 \xrightarrow{\text{send}} \text{SENT} \xrightarrow{\text{receive}} \text{RECEIVED} \\
 \xrightarrow{\text{decode.2 or decode.3}} \text{DECODED} \xrightarrow{\text{output}} \text{SENDER}.
 \end{array}$$

The new plant inserts $\text{shift}T$ and state T between Z and encode ; the supplied transfer maps identically named old states to new ones. All ordinary events are controllable. The old safety requirement forbids decode.3 ; the new one forbids decode.2 . The R1 interval forbids ten ordinary labels between old-stop and new-start. Its initializer and source interpretation are derived below.

The new requirement’s physical observer is initially zero, becomes one on decode.2 , and becomes zero on every other ordinary label. The saved lookup maps observer zero to non-error monitor state zero and observer one to error (outside the safe initializer domain). In monitor state zero, decode.2

enters error; every other label preserves zero. Update commands and transfer preserve the physical observer. In particular, start reads its current value and does not reset it.

Proposition F.3 (GSM activation with ordinary events). *The supplied GSM/R1 contract admits a state-based completing policy from every old entry whose new-decode requirement is active during ordinary decode.3 and output events. At these activations, the supplied new-requirement initializer is exact for the activation-scoped requirement $\Box \neg \text{decode}.2$; the policy reaches a compatible supplied new-controller state within twelve update events.*

Proof. While old-stop is pending, follow the old cycle only until the first RECEIVED state, then select old-stop and new-start consecutively. Select *decode.3*, then *output*, then the identity transfer at SENDER; disable other controllables. No UC event occurs and each selected label has a singleton outcome.

Every old plant state reaches the first RECEIVED in zero to seven edges without taking a decode edge. This prefix preserves the old requirement and the initially false R1 gap guard. At RECEIVED, the previous *receive* has set the observer to zero; the same holds for an old entry at RECEIVED. Hence the new initializer is defined. Stop and start preserve that observer, and no ordinary event occurs while the R1 gap guard is true. After start, both *decode.3* and *output* preserve monitor state zero. A choice of *decode.2* at that same plant state would enter error, so the active requirement constrains an actual ordinary choice.

Following *output*, the physical state is old SENDER with observer zero. Transfer yields new SENDER with observer zero and new monitor zero, exactly the supplied new endpoint's initial projection; all three commands are consumed. The remaining safe R1 monitor is of interval type and is not matched against an endpoint requirement. The fixed new controller subsequently excludes *decode.2*. Its monitor therefore recognizes precisely the safe active suffixes; violations before activation have no bearing on this ACT claim.

Let d be distance along the old cycle to the first RECEIVED. Give pre-stop states rank $d+5$; give the states after stop, start, *decode.3*, *output* and transfer ranks 4, 3, 2, 1, 0, respectively. Every edge strictly decreases rank, with maximum twelve and rank zero at a matching goal. The pending-command set and active monitors distinguish these phases, so no extra history memory is needed. This proves all-entry safety and finite completion. The same event sequence also places both boundaries before transfer, as required by the separately supplied R2 order; no R2 solver result is inferred here. $\square$

An old execution can already have performed *decode.2* before the update request. This construction thus does not establish H or safe full histories for A. It establishes a non-vacuous ACT obligation over ordinary update actions and the fixed endpoint continuation, within this supplied model. It does not validate the compiler, application intent or runtime adapter.

From the source interval to an entry guarantee. GSM/R1 requires $\Box(G \Rightarrow \neg A)$: G starts at old-decode stop and ends at new-decode start; A denotes ten decode/encode, communication and shift events. Every old entry has $G = \text{false}$, since no command has occurred. Entry preserves this guard and resets action predicates. Its monitor has guard-false, guard-true and error states: stop sets the guard, start clears it, and A enters error while the guard holds. Installing guard-false therefore gives exactly the fresh E obligation at every old entry.

The source/export check covers this formula, initializer, all 45 state-label pairs and both action-predicate entry values. This establishes declared E meaning; application intent and runtime conformance remain unvalidated. Technical Appendix E retains the full check population and older unverified results.

G Proof of Conditional Trace Lifting

Linking the phases. The linked model connects disjoint old/update/new phases by entry $z \mapsto \text{Init}(z)$, policy edges Succ_σ and handover $q \mapsto \kappa(q)$. Here $\text{Init}(z)$ is the admitted update state copied

and initialized from old state z , Succ_σ gives the policy successors, and CL^v is the monitored closed loop of endpoint version v . Entry and handover add no game event. Let α be a phase-disjoint observation map.

A1 (endpoints): actual endpoint observations map to same-labeled edges of CL^v and preserve its component and monitor states.

A2 (update observations and progress): at a non-goal, filtering the complete environment event set disables every controllable event outside $\sigma(q)$ and disables no uncontrollable event; every remaining event/outcome maps into $\text{Post}(q, a)$ and is reported with its successor identity. The resulting set is nonempty and a next event is eventually observed. This progress premise assumes no fairness among event choices.

A3 (commands and observation): each command starts at uncontrollable quiescence, finishes atomically and failure-free in finite time, and reports one modeled transfer or monitor-lifecycle transition with its successor identity. Internal τ steps, unobservable outcomes and divergence are outside the model.

A4 (boundaries): entry atomically observes the actual reachable old state z and switches to $\text{Init}(z)$; at a quiescent goal, ordinary controllables are disabled and handover installs its selected controller state and switches control. Each boundary is a finite, failure-free linearizable operation preserving the specified component and monitor states [16].

For the history-based interpretation below, let $H_r^{\text{exec}}(\xi)$ contain the actual activation histories at ξ , using the declared reference origin and event projection of r (and consuming a new requirement's start exactly once). *History completeness* is the explicit premise $H_r^{\text{exec}}(\xi) \subseteq H_r(\xi)$ at every reached activation. It is not established by A1–A4 or by synthesis.

Theorem G.1 (Conditional trace lifting). *For a returned WIN certificate $(\sigma, \text{rk}, \kappa)$ of an eligible contract, every execution satisfying A1–A4 maps through α to the linked model. From an observed entry $\alpha(x_0) = \text{Init}(z) \in Q_0$, the update reaches quiescent handover within $\text{rk}(\text{Init}(z))$ observed game events and then hands control to the compatible endpoint. With history completeness and RS for a supplied initializer, its active interval satisfies the declared residual interpretation. Agreement with application intent remains an input obligation.*

Proof. Fix an execution satisfying A1–A4 and a returned certificate $(\sigma, \text{rk}, \kappa)$. We use the phase-disjoint observation map α from the theorem; the assumptions require it to preserve the specified component and monitor states, not merely event names. A1 maps the old prefix to same-labeled edges in the monitored old closed loop. At entry, A4 supplies one linearization point at which the actual reachable old state z is observed and control changes to $\text{Init}(z) \in Q_0$. Entry contributes the linked model's boundary edge and no game event. Thus the update observation starts in the certificate domain.

Suppose the current observation is a non-goal certificate state q . A2 filters controls according to $\sigma(q)$ while retaining every enabled UC event. Its next reported event and successor identity therefore give an edge $q \xrightarrow{a} q'$ in the complete policy successor relation. For a command, A3 additionally supplies the required UC quiescence, finite atomic completion and exact modeled successor. Ordinary reports satisfy A2 directly. The checked WIN certificate includes every such successor, so q' remains safe and in its domain, with $\text{rk}(q') < \text{rk}(q)$. This establishes the update-prefix simulation and rank invariant by induction, for every outcome rather than a selected favorable one.

A2 makes a next event eventually observable at every non-goal; A3 rules out a nonterminating command between observations. Consequently the execution cannot stop or stall at a non-goal. An infinite sequence of update events would give an infinite strictly descending sequence of natural ranks, which is impossible. At most $\text{rk}(\text{Init}(z))$ game events therefore reach a goal. This is an event bound: the assumptions provide no uniform bound on the time between observations. They also impose no fairness on which enabled event or outcome occurs.

The goal predicate supplies UC quiescence and a compatible reachable new-endpoint state $\kappa(q)$. A4 disables ordinary controls, preserves the specified component and monitor states, and completes handover at a finite linearization point. Representing this operation by the linked boundary edge adds

no game event. A1 then maps every new-phase observation to the selected monitored endpoint's same-labeled continuation. Thus the complete observed execution maps into the linked model. The argument assumes the two boundary implementations have the A4 properties; it does not prove their linearizability or atomicity from implementation code.

Finally, certificate safety preserves each supplied monitor while it is active. Fix a reached activation at ξ , and an actual history $h \in H_r^{\text{exec}}(\xi)$ measured from the declared reference origin using the requirement's event projection. The explicit completeness premise $H_r^{\text{exec}}(\xi) \subseteq H_r(\xi)$ gives $h \in H_r(\xi)$. Monitor safety gives $w \in L_r(\iota_r(\xi))$ for every finite observed active suffix w . RS then gives $hw \in S_r$. The same argument applies at every activation; prefix closure covers every finite prefix of an infinite active interval. It does not establish completeness of the supplied histories or their agreement with intent. If a history-inclusive interpretation contains an earlier violation, an activation-safe scope that omits it does not satisfy this premise. Cell's activation duty and the stated ACT cases instead have their separate direct interpretation arguments in Section 4.2. Unobservable transitions, divergence, failed commands and implementations violating A1–A4 are outside the theorem. $\square$

H Endpoint Interfaces and Certificate Reuse

Let T project loadable targets onto component and new-monitor states, omitting controller state. The update-game construction uses endpoint controllers only through entries Q_0 and T .

Theorem H.1 (Endpoint-interface reuse). *Keep components, transfers, monitors, initializers, event control and precedence fixed. Eligible contracts with equal entries and target projections have identical update games. Every WIN policy/rank and LOSS certificate transfers between them; selecting a compatible target of the second contract preserves that endpoint's continuations. A winning policy also transfers to fewer entries and more projected targets, stopping at newly admitted goals.*

The following proof establishes each clause. This interface result does not claim an implemented cache or a measured reuse speedup. Let K and K' satisfy Theorem H.1. Their local data define the same finite ambient state space of version-tagged component tuples, monitor memories and pending-command sets. Let I, I' be their entry sets, and let T, T' be their target projections. A target projection retains all component and new-monitor coordinates and removes only the endpoint-controller coordinate. All targets retain their eligibility premises, including reachability and endpoint safety, deadlock freedom and preservation of uncontrollable events.

Identical interfaces. Assume $I = I'$ and $T = T'$. On every ambient state, enabled ordinary events and their complete outcome buckets are equal because the component models and controllability agree. Transfer guards and outcomes agree because the transfer relations, pending commands and precedence agree. Boundary guards and memory updates agree because monitors and initializers agree. UC quiescence and safety therefore agree as well. Induction on path length from I gives equal reachable state sets and label/outcome transitions. No rule inspects either endpoint-controller state after entry. Old states collapsed by Init may have different controller coordinates, but such coordinates impose no source restriction once old control is released.

A reached state is a goal precisely when it is safe, UC-quiescent, has no pending command, and its component/new-monitor projection belongs to T . Hence goals agree. The identity map preserves the policy successor relation, safety, nonblocking, all maximal paths and every strict rank inequality. It also preserves the losing-certificate obligations: goals are absent from L , a losing entry remains in L , and at each safe member the same UC escape witness or all-control counter-outcomes remain in L . Thus WIN and LOSS certificates transfer, including their universal quantification over all enabled outcomes.

For every reached goal q , membership of its projection in T' supplies at least one full target $z' \in Z'_{\text{load}}$. Choose one and define $\kappa'(q) = z'$. The selected endpoint controller is eligible and the handover copies precisely its component, monitor and controller coordinates. Proposition 3.2 yields an isomorphism to that endpoint rooted at z' . This preserves that endpoint's complete continuation language;

it does not claim equivalence between the two endpoint controllers' continuation languages. Nondeterministic local outcomes are unchanged and remain adversarial. Several targets with one projection cause no difficulty: selecting a compatible controller state is part of handover, not selection among an event's nondeterministic outcomes.

A Cell replacement with different continuation behavior. Delete only $c_0 \xrightarrow{j} c_0$ from the new controller in Table A1. Its alphabet still contains j through $c_1 \xrightarrow{j} c_0$. The reachable endpoint tuples remain exactly $z_A = (h, e, c_0, c)$, $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$: the cycle $z_A \xrightarrow{m_B} y_B \xrightarrow{j} z_B \xrightarrow{m_A} z_A$ remains, and the deleted edge removes only the inspection self-loop at z_B . This closed loop remains safe and deadlock-free; all events are controllable. Keeping $Z_{\text{load}} = \{z_A, z_B\}$ preserves its target projection, and the old endpoint is unchanged. Thus Figure 1's whole update certificate and its handover choices transfer. The endpoint languages differ: from z_B , the original admits j^ω , whereas the replacement's first event must be m_A . Reuse therefore does not require continuation-language equivalence between endpoints. This finite construction adds no solver measurement.

Smaller entry interface and larger target interface. Assume instead $I' \subseteq I$ and $T \subseteq T'$, and let $(\sigma, \text{rk}, \kappa)$ win for K . Starting from I' , use σ until the first goal of K' and stop there. Every resulting nonterminal prefix is also a prefix of the original policy from I , because all local transitions agree. Safety and nonblocking are inherited. An infinite resulting path would be an infinite original-policy path, contradicting its finite completion. Every original goal has its projection in T' and is also a goal when reached from I' , so a maximal resulting path cannot pass an old goal without stopping. A new goal may be reached earlier, which only truncates a path.

Restrict rk to this truncated closure and assign zero at its goals. Edges between remaining non-goals keep their original strict inequalities. An edge entering a goal has a positive-rank non-goal source and zero-rank target, so also decreases. Choose a compatible target in Z'_{load} at each such goal. This supplies a complete WIN certificate for K' . The converse is not claimed: fewer entries or additional targets can make a formerly losing contract win. Nor does the theorem preserve the particular policy returned by a new solver run, which may use a different endpoint-dependent exploration order.

H.1 Exactly When a Fixed Returned Policy Can Be Reused

The following result is relative to an already returned WIN certificate; it does not require equal or nested target interfaces.

Theorem H.2 (Policy reuse). *For a returned WIN certificate $(\sigma, \text{rk}, \kappa)$, write $D = \text{Reach}_\sigma(Q_0)$ and $G_D = D \cap Q_{\text{goal}}$, and extend σ by the empty set on G_D . Fix components, transfers, monitors, initializers, event control and precedence. For another eligible endpoint pair with entries $Q'_0 \subseteq D$ and goals G' , let D' be the closure under the extended policy, stopping at G' . That policy wins exactly when $D' \cap G_D \subseteq G'$.*

Here is the complete construction and proof.

Let K have certificate $(\sigma, \text{rk}, \kappa)$, finite support $D = \text{Reach}_\sigma(Q_0)$ and terminal set $G_D = D \cap Q_{\text{goal}}$. Define $\bar{\sigma}$ on D by retaining σ off G_D and choosing no controllable at G_D . The latter states have no UC event by the original goal condition, so they have no successor under $\bar{\sigma}$. Every state of D is safe. Each original nonterminal has at least one successor, retains every UC and every selected-label outcome within D , and has strictly larger rank than each of those successors.

Let K' be eligible with the same components, transfers, monitors, initializers, controllability and precedence, but possibly different endpoint controllers and loadable targets. Assume $Q'_0 \subseteq D$. After admission the successor rules inspect only the fixed local data, so they agree on D in both contracts. Evaluate the new goal test G' on these states using the new projected targets. Starting from Q'_0 , form the least closure D' under $\bar{\sigma}$, expanding only states outside G' . Closure of the old certificate gives $D' \subseteq D$. In particular, constructing D' requires only the stored policy successor graph and the new

entry/goal information; it does not explore unselected update actions. Obtaining and validating that endpoint information remains a separate task.

Suppose $D' \cap G_D \subseteq G'$. At a reached non-goal, the state cannot be in G_D , so it retains the original nonempty successor set, with all uncontrollables and selected-event outcomes. Every such successor is in D' . Safety is inherited from D . Set $\text{rk}'(q) = 0$ on $D' \cap G'$ and $\text{rk}'(q) = \text{rk}(q)$ elsewhere. An original nonterminal has positive rank: otherwise its nonempty successor set would require a negative natural-number rank. Thus every edge entering a new goal strictly decreases, and every remaining edge retains its original strict inequality. No infinite non-goal run exists, and no finite maximal run can end at a non-goal. The policy therefore wins from every new entry. For each reached new goal, choose any compatible full target in Z'_{load} to define κ' . Its existence follows from the new goal test, and endpoint eligibility plus Proposition 3.2 preserves that endpoint's own continuation language.

Conversely suppose $q \in D' \cap G_D$ but $q \notin G'$. By the definition of D' , a finite run from a new entry reaches q without having stopped at a new goal. As an original terminal, q is UC-quiescent, and $\bar{\sigma}(q) = \emptyset$. Hence this is a finite maximal run ending at a non-goal deadlock in K' , so the fixed policy is not winning. This proves both directions. It says nothing about another policy or about extending the old policy beyond its former terminals; either may solve K' .

A simple sufficient target condition. If the new target projection contains the projections of every member of G_D , every original terminal remains a new goal. The criterion then holds for any eligible $Q'_0 \subseteq D$. Unlike whole-interface equality or target-set enlargement, it permits removing old loadable targets never used by the certificate. It is only sufficient: new goals can also stop every path before a removed old terminal.

Application to the saved PC2 certificate. The PC2-Rolling certificate used in Technical Appendices K and K.1 has 126 entries, 381 states and 522 edges, with maximum rank 37. Its only all-commands-completed state is its single terminal, whose handover selects new-00000009. Keeping the old endpoint and narrowing the loadable set to this one reachable target leaves that same certificate valid, even if the original loadable set contained other targets. This is a direct specialization of the sufficient condition, using the already saved certificate; it is not a new synthesis or timing result. The eligibility of a different supplied new controller, and the presence of a target with the required component and all-new-monitor coordinates, must still be checked. No equivalence between different endpoint controllers' continuation languages is assumed.

I Complete Native DUCS Construction for Cell

This section specializes main-paper Table 1 to a complete native DUCS encoding of Cell; Appendix J gives the GR(1) encoding. Table A4 maps each Cell obligation to its encoding. These chosen constructions require justified input meanings and establish neither a general translation nor minimum-size, effort or performance results.

We construct a correct Cell update using native DUCS environments, relation R , transition requirement T and boundary protocol O [31, Defs. 4.1–4.3 and 5.1].

DUCS's `hotSwap` event requests the update and interrupts old control. Its `reconfigure` milestone switches environments; our extra h connects mixed stages to a fixed endpoint.

Environments and entry. Refine the old controller to its reachable closed loop $c_A \xrightarrow{m_B} c_B \xrightarrow{m_A} c_A$, preserving ordinary traces and recording the holder. The old environment has these same moves. Let E' contain eight stages $(v_A, v_B, L) \in \{o, n\}^2 \times \{A, B\}$ and three post-handover states. In a stage, m_B moves A to B , m_A moves B to A , and j self-loops only if $v_B = n, L = B$. Transfer ρ_i changes $v_i = o$ to n only while station i is empty. A controllable h maps (n, n, L) to z_L . Post-handover edges copy the

fixed new closed loop:

$$z_A \xrightarrow{m_B} y_B \xrightarrow{j} z_B, \quad z_B \xrightarrow{j} z_B, \quad z_B \xrightarrow{m_A} z_A.$$

Here y_B is inspection-pending; z_A, z_B are clear targets. The standalone initial state is z_A , but interrupt entry uses $R : L \mapsto (o, o, L)$. The 11-state count excludes protocol and monitors and is not a minimality result.

Requirements and memory. Write $s = \text{startNewSpec}$ and $t = \text{stopOldSpec}$. Protocol O accepts these physical stutters. Let $\dot{a}$ record last event a , resetting on every other label. Fluents $B = \langle m_B, m_A, \text{false} \rangle$ and $P = \langle m_B, j, \text{false} \rangle$ retain holder and uninspected-arrival history. Take $G = \neg \dot{j}$ and $G' = \neg(P \wedge \dot{m}_A)$ under native old-until-stop and new-after-start scopes.

For each event a , let S_a be its initially false, never-reset occurrence flag. Take T to be the conjunction

$$\begin{aligned} \Box(\dot{s} \Rightarrow S_{\text{reconfigure}}), & \quad \Box(\dot{t} \Rightarrow S_s), \\ \Box(\dot{h} \Rightarrow S_{\rho_A} \wedge S_{\rho_B} \wedge S_s \wedge S_t \wedge \neg(B \wedge P)), & \quad \Box(\text{reconfigure} \Rightarrow \Diamond \dot{h}). \end{aligned}$$

Interrupts, O , version guards and the post-handover region ensure at most one occurrence of each update event. This analytical T includes eventual h , beyond the published safety- T implementation [31, Sec. 5.3]; no solver experiment is claimed. Native DUCS still requires eventual stopOldSpec, reconfigure and startNewSpec, including in its safety- T implementation. Here reconfigure enters the mixed stages; h is an additional fixed-target connection, so those three milestones do not by themselves require h . We do not rule out another encoding within safety- T .

Proposition I.1 (Cell witness preservation). *This staged native DUCS instance admits a correct update from both reachable old entries. On the segment from hotSwap through h , erasing these two events and reconfigure and renaming s, t to Cell's boundaries gives the displayed FG policy paths. After h , its ordinary traces equal those of the supplied new closed loop rooted at the selected target.*

Proof. Use DUCS's assembled environment, including O , its old-event renaming/fluent extension, and controller extraction [31, Sec. 5, Theorem 5.1]. Preserve old behavior before hotSwap. Map $H(c_L)$ to the corresponding pre-reconfiguration controller state; then choose reconfigure, the six-/five-event path of Figure 1, h , and every fixed-endpoint edge thereafter. All choices are enabled in the assembled environment; no ordinary UC event remains after initiation, ensuring legality and deadlock freedom.

Old j is disabled until t . On safe active prefixes the correspondence is: the FG monitor is pending iff $B \wedge P$. At s , a B holder has an earlier m_B and no j , so both are pending. At an A holder, B is false and FG is clear, although P may remain true from an earlier visit to B . This is equivalence of permitted continuations on the plant, not equality between P and the monitor state: m_A and j are disabled at A , and the next m_B makes both representations pending. Thereafter j clears P and the monitor. An enabled m_A comes from B ; it leaves P unchanged while setting $\dot{m}_A$, so it violates G' exactly when FG was pending. Otherwise it leaves $B \wedge P$ and FG clear. Transfers and boundaries preserve the correspondence. Thus every ordinary extension has the same safety outcome in both representations. The finite paths satisfy the boundary order and reach all-new, clear targets, fulfilling T and the native update milestones. After h , the synchronized environment/controller copies retain every fixed-endpoint edge. Their reachable product is diagonal, giving the claimed ordinary-trace equality. $\square$

The native encoding explicitly supplies stages, guards, holder refinement, history fluents, target gate and post-handover graph. FG-DUCS generates the update-game counterparts from local transfers, lifecycle rules, initializers and fixed targets, returning rank and handover certificates. This deterministic, fully controllable example establishes one-direction witness preservation, not a general translation, solution-set equivalence, measured effort saving or performance advantage.

Comparing the supplied structure. We count the same semantic objects in the three displayed Cell constructions (Appendices B, I, J). Their shared reference endpoint graphs have two states/two edges (old) and three states/four edges (new), with two load targets. Table A4 tracks how the same Cell objects are encoded. Its rows are not additive input-size or effort measures.

Table A4: The same Cell objects through three explicit input constructions. Counts describe these constructions, not minimum encodings.

Common object	FG-DUCS	Chosen DUCS input	Chosen GR(1) input
8 version/holder configurations	Generated one-workpiece physical projection of the local component product.	8 mixed stages supplied in E' .	8 valuations of supplied coordinates h, v_A, v_B .
2 empty-station transfers	Two supplied singleton relations g_A, g_B ; guarded commands generated.	Two transfer labels with version/empty-station guards in E' .	Two guarded-assignment rows in R_B .
2 requirement boundaries and 1 order	OLD/NEW types and supplied $s \prec t$ generate boundary guards and active lifetimes.	Native s, t , with their ordering clause in T .	Supplied bits s, t and the stop row's ordering guard.
Inspection duty at start	Supplied holder-dependent initializer installs pending iff B holds.	Supplied history fluents B, P and G' retain the corresponding duty.	Supplied start assignment $p' = [h = B]$.

DUCS generates its standard protocol O and interrupt composition; GR(1) generates its bridge and switch bookkeeping. These are not extra modeler inputs.

The modeler still supplies FG-DUCS with three monitors: eight states and eight explicitly listed nondefault edges in Table A1, before adding default self-loops and absorbing-error edges. The other constructions encode the one-workpiece invariant directly in the holder coordinate. Thus local generation does not remove all modeling obligations or show fewer supplied objects. The DUCS target gate and post-handover graph retain all four endpoint edges; the GR(1) post relation permits them, but its synthesized C_2 may retain fewer. FG-DUCS instead takes the fixed controllers and targets as inputs and generates their matching tests. The demonstrated difference is the generation of mixed stages and lifetime operations from the local contract, not a measured effort or size advantage.

J Cell through a GR(1) Bridge Interface

Amram et al.'s update problem takes old/new GR(1) specifications and a switching condition. Before switching, an update strategy satisfies the old system safety relation; afterwards it satisfies the new relation and new justice, under the new environment safety assumptions [2, Def. 3.1]. Their construction generates the bridge and its *switch/allowed* bookkeeping [2, Sec. 4.1]. We describe concrete predicates for Cell to identify the additional mixed-version and lifetime structure supplied by this Cell construction. This is an analytical input construction, not a Spectra run or a general translation.

A phase extension of the endpoint specifications. Cell is deterministic and all its events are controllable. Take no environment variables and the environment safety relation to be true. System variables are the holder $h \in \{A, B\}$, versions $v_A, v_B \in \{0, 1\}$, new-start and old-stop flags s, t , inspection debt p , a phase $d \in \{\text{bridge}, \text{post}\}$, an event selector a , and the post-phase controller coordinate $c \in \{c_0, c_1\}$. Before start, $p = 0$. The old initial valuation has $h = A$, all five bits zero, $d = \text{bridge}$, $c = c_0$ and an unused initial selector value; its other old entry has $h = B$. The two-state old closed loop only alternates m_B, m_A and preserves all added bits.

Let $G = v_A \wedge v_B \wedge s \wedge t \wedge \neg p$. Define R_B on bridge-phase states with $\neg G$ by the following guarded assignments. A row sets a' to its label and performs its assignments; every other unlisted coordinate is unchanged. There is no additional waiting round.

Label	Guard	Assignment
m_A	$h = B \wedge (\neg s \vee \neg p)$	$h' = A$
m_B	$h = A$	$h' = B, p' = s$
j	$h = B \wedge v_B \wedge t$	$p' = 0$
ρ_A	$\neg v_A \wedge h = B$	$v'_A = 1$
ρ_B	$\neg v_B \wedge h = A$	$v'_B = 1$
start	$\neg s$	$s' = 1, p' = [h = B]$
stop	$s \wedge \neg t$	$t' = 1$

The holder type preserves exactly one workpiece; transfer guards express empty-station replacement; $s \wedge \neg t$ enforces start-before-stop. The j guard releases the old inspection prohibition only after local stop. Thus R_B is a *phase extension* of the old specification, not the unmodified old requirement. Keeping the original global prohibition until the final specification switch would exclude the displayed path that inspects before its last transfer. The supplied old controller nevertheless satisfies this extension because it never changes the added bits or inspects.

Take the old system safety relation to be R_B and the switching condition to be $d = \text{bridge} \wedge G$. The new system relation permits one auxiliary handover from that condition, preserving h, v_A, v_B, s, t, p and setting $d' = \text{post}, c' = c_0, a' = \text{handover}$. In the post phase it permits exactly the endpoint edges

$$z_A \xrightarrow{m_B} y_B \xrightarrow{j} z_B, \quad z_B \xrightarrow{m_A} z_A, \quad z_B \xrightarrow{j} z_B.$$

Here $p = 0, 1, 0$ and $c = c_0, c_1, c_0$ at z_A, y_B, z_B , respectively, and both versions and lifetime flags are one. The new initial valuation is post-phase z_A ; both justice conditions are true. The two loadable bridge projections reach this new winning region through the auxiliary handover.

What this construction establishes. Each edge of Figure 1 satisfies its row: start records the actual holder, inspection clears debt only after old-stop, and transfer preserves debt. Both displayed paths reach G and then take the auxiliary handover. With FG rank r , assign $r + 1$ before handover and zero afterwards; every bridge round, including the auxiliary handover, strictly decreases rank. The six-/five-event paths therefore give seven-/six-round bridge witnesses. Erasing the auxiliary handover preserves their local events, initialized memories and selected physical targets. Unsafe controllable choices are excluded by the guards; we assert witness preservation, not equality of unsafe game buckets.

The post relation permits every supplied endpoint edge, although synthesizing a new GR(1) controller may select fewer of them. Once that C_2 is selected, Amram et al. execute it unchanged after the bridge [2, Sec. 4.2, Alg. 2]. FG-DUCS instead takes the replacement controller and loadable targets as inputs; its continuation guarantee concerns that supplied controller. GR(1) predicates also avoid enumerating every mixed state: the comparison concerns who supplies the version, lifetime, initialization and target equations, not a state-count lower bound. FG-DUCS derives those equations' operational counterparts from the local contract. No modeling-effort or runtime reduction is inferred.

The construction does not extend unchanged to contracts with UC or nondeterministic outcomes. Their environment/system move order and observation of realized successors must be aligned: replacing a control choice followed by all its outcomes with an environment-first round can change $\exists a \forall o$ into $\forall o \exists a$. The complete deterministic Cell comparison avoids that issue; the main all-outcome guarantee remains defined by FG-DUCS's own game.

K Saved PC2 Activation Histories and Their Boundary

The supplied old controller and pre-synthesized new controller are fixed. Baseline PC2's old requirements and 12 new requirements remain; added new-endpoint calibration availability yields 13 new starts, separate from the interval duty. Transfers map old destinations to calibration counterparts, retaining ordinary observer memory. Work, calibration requests and updates are controllable; arrivals, tool outcomes, resets and calibration completion are UC. Entry inherits old memory and initializes the

interval monitor; new-start looks up the current ordinary observer tuple. No precedence is supplied. Targets match components and new-monitor memory at reachable new-endpoint states.

Interval R_{CAL_READY} observes transfers as calibration entry; endpoint availability observes ordinary requests. Completion restores both. Handover matches component, observer and new-monitor memory; real-history and runtime obligations remain unvalidated.

Generation responsibilities beyond Cell. PC2 separates an application obligation from its generated update protocol. The designer supplies the two old/new arm models, their local transfer maps, twelve old and thirteen new requirements, the interval duty and fixed endpoints. For interval availability, each readiness bit starts true, becomes false on its arm’s transfer or calibration request, and becomes true on completion; their disjunction must hold. This obligation is supplied, not inferred from the plant.

From these inputs, FG-DUCS constructs the twenty-five independent requirement boundaries and two local transfers, their active-monitor updates, mixed-version successors, UC-quiescence guards and target tests. The frontend prepares current-observer lookup maps and physical closure; the lazy solver generates the update product on demand. The saved result covers 126 entries with 381 policy states, 522 edges and maximum rank 37. Thus neither the sequence nor those policy states are part of the input. These counts describe one saved result, not a bound on the full mixed product.

For the published DUCS interface, a representation of these independently changing arms and requirement memories would place their staged behavior and admissibility in the supplied E' and transition requirements. DUCS itself still generates its global update protocol; that work is not assigned to the designer [31, Secs. 4.2, 5]. A GR(1) representation would supply corresponding version, local-state, active-memory and observer variables with transition constraints; its existing construction generates switching bookkeeping and the bridge [2, Def. 3.1, Sec. 4]. Unlike the deterministic Cell construction, PC2 has UC outcomes, so this responsibility comparison does not establish an equivalent GR(1) game or a general translation. It identifies which local operations FG-DUCS generates directly, without measuring modeling effort, minimum encoding size or external-solver performance.

Table A5: One saved 31-event PC2-Rolling path; other entries can follow different command orders. O/N: old/new; ARM: unprocessed workpiece; CAL: its calibration counterpart. Ordinary observers stay zero.

Reached after	Arm 1	Arm 2	Available	Rank
Update entry	O/initial	O/initial	2	31
Two arrivals, twelve old stops	O/ARM	O/ARM	2	17
Transfer arm 1	N/CAL	O/ARM	1	16
Calibration completion 1	N/ARM	O/ARM	2	15
Transfer arm 2	N/ARM	N/CAL	1	14
Calibration completion 2	N/ARM	N/ARM	2	13
Thirteen new starts: handover	N/ARM	N/ARM	2	0

At handover, all 13 new monitors are 0 and no command remains; the matched endpoint has both arms operational. Certificate identifiers are in Technical Appendix K. This path supplies no timing and does not replace all-entry/all-outcome checking.

This analysis traverses existing PC2-Rolling exports; it runs no solver and adds no timing trial. The selected requirement is $\Box(out_1 \Rightarrow Drilled_1 \wedge Cleaned_1 \wedge Painted_1)$. The three fluents start false, successful tool events set them, and $reset_1$ clears them. An out_1 with any false fluent violates the requirement. Other update commands stutter for these fluents.

The saved old closed loop contains 126 reachable states and 348 edges; the policy contains 381 states and 522 edges. Its maximum state and entry rank is 37; all 522 edges decrease rank, and the longest entry-to-goal path has 37 events. The stored certificate therefore covers all 126 entries; the rank-31 path below is one representative. Matching component states, old-monitor memory and ordinary observers identifies each policy entry with one old state. Every maximal policy run crosses the single

selected start edge $q_{13} \rightarrow q_{12}$, which installs monitor state 0 at two NEW/ARM components with all 24 ordinary observers zero. Ranks strictly decrease; removing that start edge makes the unique goal unreachable from every entry. The path in Table A5 starts at q_{126} and ends at q_7 , whose handover target is new-00000009. All 13 new monitors are 0 there and no command remains. Its prefix to q_{12} has two arrivals, twelve stops, four transfer/completion events and eight starts, with no out_1 or successful tool result.

For each entry, find an old prefix from the declared old initial state and a policy prefix ending at this start. Keep an edge only if the source fluent values and its label do not violate the selected new requirement. Forward reachability in the old graph and reverse reachability from the start in the policy graph yield the table. Separate per-entry breadth-first traversals construct positive witnesses or record failure of each portion; set-based fixed points reproduce the same partition.

Safe old prefix exists	Safe policy prefix exists	Entry count
Yes	Yes	39
No	Yes	21
Yes	No	66

All 126 entries reach the start through some unrestricted saved-model history. The 39 positive rows establish existential nonemptiness at this one reached tuple, not safety of all histories. An old cycle

$in_1; drill_1; drillOk_1; polish_1; polishOk_1; clean_1; cleanOk_1; out_1; reset_1$

returns to the identical old initial tuple. It violates the new requirement at out_1 because $Painted_1$ is false. Prepending this cycle to any entry’s history preserves its final activation tuple while retaining a past violation. The new monitor is absent in every old state and all 375 policy ancestors of its start, so none of these negative histories refutes its active-lifetime safety. They show why activation-safe and history-inclusive scopes cannot be conflated.

Replay of source-declared plant moves and all 24 observer updates agrees with all 348 old and 522 policy edges. This establishes consistency of these saved histories, not independent frontend correctness or completeness of exported outcomes. The artifact retains all 126 rows, positive paths, negative reasons and 18 boundary edges separating the safe reachability sets. Only one reached tuple of one requirement is covered; the reported 38,288 initializer guard tuples, the other 12 PC2 new requirements and the 138-occurrence population are not checked for nonemptiness by this analysis.

Direct ACT argument after the selected start. ACT begins the invariant with the actual observer values at activation. For the selected requirement these values are all zero. The saved policy graph’s complete suffix is:

Source (rank)	New requirement started	Target (rank)
q_{12} (5)	OUT_IF_FINISHED_2	q_{11} (4)
q_{11} (4)	TOOL_ORDER_1	q_{10} (3)
q_{10} (3)	TOOL_ORDER_2	q_9 (2)
q_9 (2)	PAINT_ONCE_1	q_8 (1)
q_8 (1)	PAINT_ONCE_2	q_7 (0)

All six states have two NEW/ARM components, 24 zero observers and selected-monitor state 0. No ordinary event, transfer or uncontrollable edge occurs in this suffix. The unique goal matches new-00000009. Consequently the source invariant holds at activation and on every remaining update step; matching the verified new endpoint supplies its post-handover guarantee.

The saved graph is closed over 381 entry-reachable states; all 522 edges decrease a natural rank, and no non-goal deadlocks. Removing the unique selected start leaves 375 reachable states and no goal from any of the 126 entries. Thus every maximal saved-policy run crosses the start and follows this suffix. The static checker `check_saved_suffix.py` reproduces these facts from the existing certificate

without invoking synthesis. Completeness of modeled outcomes relies on the stored certificate checker; frontend/compiler correctness and real execution are not independently proved here. The argument concerns one requirement and one reached tuple, not all 138 NEW occurrences. The A/H diagnostics above remain unchanged.

K.1 All New Requirements of the Saved PC2 Policy

The previous history analysis fixes one requirement to distinguish A from H. A separate inspection of the same saved policy establishes the active-lifetime obligation for all thirteen new requirements. It introduces no solver run, input change or timing trial.

The only new-start edges are the chain $q_{20} \rightarrow q_{19} \rightarrow \dots \rightarrow q_7$, in this order: stamping avoidance for arms 1 and 2; calibration availability; clean-once for arms 1 and 2; drill-once for arms 1 and 2; output-if-finished for arms 1 and 2; new tool order for arms 1 and 2; and paint-once for arms 1 and 2. The selected start above is the eighth edge. Every state in this chain has two NEW/ARM components, all 24 ordinary observers zero and rank $q - 7$ at q . Each newly installed monitor is 0 and stays 0. There is no ordinary event, transfer or UC edge after the first new-start.

Proposition K.1 (All PC2 active obligations). *Every maximal run of the saved PC2-Rolling policy from any of its 126 entries activates all thirteen new requirements and satisfies each source invariant from its own activation through handover. The matching verified new endpoint preserves these obligations afterward.*

Proof. The saved domain is entry-reachable and closed, every edge strictly decreases a natural rank, and no non-goal deadlocks. Hence every maximal run reaches the unique goal q_7 . Every q_j , $7 \leq j \leq 19$, has only its preceding chain edge as an incoming policy edge, and no entry lies on the chain. Thus every run traverses all thirteen starts.

For each arm, the source invariants forbid stamping; forbid cleaning, drilling or painting an already completed workpiece; require all three completed-tool fluents at output; and constrain tool-pending fluents by the new tool order. At activation all completed-tool and pending fluents are false, and no ordinary action occurs on a start label. Each invariant therefore holds. The calibration invariant is $\Box(\neg \text{Calibrating}_1 \vee \neg \text{Calibrating}_2)$. Its source fluents start false and are set only by ordinary calibration requests. The old plant has no such request, so every old entry retains false values; none occurs in this policy either, while completion clears them. They too are false on the start chain. Start commands preserve these fluents and produce no ordinary action, so all active source invariants continue to hold until q_7 . This goal matches new-00000009, with both components, all ordinary observers and all thirteen new monitors aligned. The supplied verified endpoint and continuation preservation give the post-handover part. $\square$

This covers all new requirements of this one saved policy at their reached activation tuples. It does not prove initializer exactness over every permitted tuple, safe pre-activation histories, all 138 NEW occurrences, compiler correctness or runtime conformance. The 39/87 history partition above is unchanged. PC2 completes calibration before activating any new requirement; its active update suffix contains only starts. The GSM construction in Proposition F.3 complements it with ordinary behavior under an active new requirement.

The earlier violations have a concrete source distinction: old PC2 requires drilling, polishing and cleaning, whereas the new output rule requires drilling, cleaning and painting. The update contract preserves its interval calibration obligation and whichever old requirements remain active; it begins each new duty at its start. This defines the specified obligation, without establishing that an operator would choose that scope.

L Exploration Order and Work Bound

The RQ3 LTS configurations disable the preliminary guided search (both guided limits are zero). The complete algorithm queries every uncontrollable candidate first. If any such bucket is nonempty, it

omits controllable queries by the UC-priority reduction. The underlying Post relation still contains any enabled ordinary controllable edges, which Direct-Full materializes. Otherwise lazy and update-first consume their sorted controllable cursor through the first nonempty bucket, deferring the rest; eager consumes every controllable candidate. A deferred candidate is never removed merely because it appears unpromising.

The default lazy/eager priority depends on the pending operations and endpoint projection, as follows. Entries in each row are tried from left to right; all listed types remain candidates if the game enables them.

Condition	Priority classes (first to last)
Some old stop pending	stop; transfer; ordinary; other update
No stop, some transfer pending	transfer; ordinary; other update
No stop or transfer, endpoint aligned	start; all other events
No stop or transfer, not aligned	events other than start; start
Update-first ablation	every update; every ordinary event

Within a class, the implementation prefers updates if update-first is configured or the endpoint projection is aligned, and ordinary events otherwise; remaining ties use type order start, transfer, stop, ordinary, then Java string order on the event label. Endpoint mismatch is the minimum over load signatures of the number of disagreeing component/new-monitor coordinates. An old coordinate agrees when its transfer contains the target; an inactive new monitor agrees when its defined activation map produces the target state.

Open is a LIFO stack. Initial roots and each newly discovered batch are sorted by increasing endpoint mismatch, then pending-set size, then canonical state order, and inserted in reverse so the least element is popped first. Existing stack entries keep their positions; this is not a global priority queue. Canonical ties compare component count, then each component’s version and state, sorted active-monitor identifiers and states, and sorted pending labels; the problem’s comparators define local state order. Already queued/expanded states are not inserted again. Retry is FIFO: append a state when it first has deferred candidates, remove from the head, skip winners/exhausted cursors, consume through the next nonempty bucket, and append again if candidates remain. Drain propagation after each expansion/resumption, and always serve Open before Retry. These rules are implemented by `FineGrainedSuccessorOracle` and `OfDucssSynthesizer`; the artifact retains their exact source. Eager changes bucket materialization and update-first changes candidate priority, providing the reported sensitivity checks without selecting an order after observing results.

Scope of the work bound. Let d count discovered states, c candidate queries (including empty results), and o returned outcome incidences over those queries. With constant-time indexed state predicates, $O(1 + |\text{Post}(q, a)|)$ successor enumeration and constant-time bookkeeping, Algorithm 1 admits an implementation with $O(d + c + o)$ work and storage. With preprocessing costs $T_{\text{end}}, S_{\text{end}}$, total bounds are $O(T_{\text{end}} + d + c + o)$ time and $O(S_{\text{end}} + d + c + o)$ storage. Entry sets can themselves be exponential in compact component descriptions. The search term counts the finite explored portion and excludes representation-dependent predicate costs. Java sorts and scans active transitions for quiescence and repeatedly checks roots; its timings do not measure constant-cost semantic queries.

Counters and charges. Intern each state once and queue its expansion once. A cursor advances monotonically through each state’s finite candidates; thus each query and each cursor/Retry transition is charged to a candidate or state. For each queried nonempty bucket, retain an outcome counter and the maximum rank of already proved outcomes. An outcome pointing to a winner updates these immediately; otherwise store one reverse incidence at its target. When a state first enters the winning set, process each of its reverse incidences once. Hence all counter decrements and maximum updates total $O(o)$.

At a state with enabled uncontrollables, combine their returned outcomes into the required universal proving set; promotion is enabled only after every UC candidate has been queried. At a quiescent

state, any nonempty control bucket whose counter reaches zero can certify promotion. Recording the first such bucket needs constant work per completed bucket. Nonempty flags prevent an empty bucket from certifying progress. A state is promoted at most once; its rank is one above its proving set’s accumulated maximum, and the retained witness supports extraction.

Maintain Open, Retry and the promotion worklist by indexed membership and removable entries, so enqueue/dequeue/removal is constant time and exhaustion needs no global scan. A remaining-root counter avoids repeated all-root scans. Expected-amortized constant-time state dictionaries suffice. Goal initialization and unsafe-state exclusion cost $O(d)$. The final policy traversal or losing-region extraction scans only stored states and incidences. Summing these charges proves the stated bound. The worklist computes the same monotone promotion fixed point as the abstract algorithm; this argument does not assert a linear running time for every implementation of its instruction to exhaust promotion.

Full-expansion containment and the complete independent-family argument are given in S2.3. Direct-Full constructs the reachable game with unsafe/goal terminals; Lazy may reach that whole graph in the worst case.

M Complete Benchmark Input Counts

Inputs have 1–5 components, 4–131 entries, 1–12 old/new requirements each and 3–26 commands, using PC2/R2’s input log.

Table A6: The nine adapted inputs. Old/new requirements are shared across variants; interval counts are Base/R1/R2.

Input	Comp.	Entries	Old/New	Interval B/R1/R2	T	U
GSM	1	8	1/1	0/1/2	8	3
Industry	3	131	3/3	0/3/6	16	9
MetaSocket	2	4	1/1	0/1/2	3	4
PowerPlant	2	10	4/3	0/3/7	4	9
PC Arms=1	1	9	6/6	0/6/12	7	13
PC Arms=2	2	126	12/12	0/12/24	14	26
Railcab	4	22	5/7	0/7/11	11	16
Surveillance	2	112	7/7	0/7/14	16	16
Workflow	5	28	5/6	0/6/11	12	16

T counts declared transfer pairs before fluent observers; U counts transfer, stop and start commands. Entries agree across completed variants; PC Arms=2/R2 times out and its requirement counts come from its input log. Only Railcab has branching transfers. These nine inputs declare no explicit precedence; the separate cell witness requires start $\prec$ stop; R2 uses interval monitors.

Requirement configurations. Each inherited input has three named requirement configurations: Base has no interval requirement; R1 protects selected actions during a requirement gap; R2 imposes selected ordering constraints. R1 blocks each designated action while its flag is true: any old-stop sets it, its designated new-start clears it. R2 requires selected boundaries before their transfers. In the GSM mobile-communication model, R1 blocks communication and decoding; R2 puts both decode-rule boundaries before transfer without ordering them. Event choices vary by model. PC2-Rolling, a production-cell calibration example, separately tests availability. The archived fork’s DUCS backend using generalized-reactivity goals (Legacy) also synthesizes subsequent control, a different objective.

N All Completed Direct-Full Exploration Pairs

The all-contract outcome summary is Table 4 in the main paper. The table below reports every completed pair.

Table A7: Absolute exploration costs for all 14 completed Direct-Full pairs. Figure A5 displays every fixed-budget contract for all four methods.

Input / variant	States (L/DF)	Queries (L/DF)	Solver s (L/DF)
GSM / Base	31 / 75	27 / 658	0.0224 / 0.0416
GSM / R1	34 / 115	44 / 706	0.0228 / 0.0450
GSM / R2	34 / 68	78 / 352	0.0201 / 0.0327
Industry / Base	272 / 282,709	3,336 / 5,058,452	0.190 / 154
Industry / R2	7,717 / 51,096	52,919 / 725,391	1.15 / 28.0
MetaSocket / Base	8 / 82	12 / 289	0.0149 / 0.0408
MetaSocket / R1	8 / 95	12 / 257	0.0136 / 0.0426
MetaSocket / R2	16 / 34	24 / 79	0.0152 / 0.0240
PowerPlant / Base	73 / 8,168	165 / 47,618	0.0463 / 1.85
PowerPlant / R1	759 / 23,686	1,927 / 66,394	0.125 / 5.51
PowerPlant / R2	1,330 / 3,594	3,118 / 13,512	0.180 / 1.01
PC Arms=1 / Base	83 / 407,019	840 / 3,842,690	0.0741 / 157
PC Arms=1 / R2	56,443 / 158,218	354,259 / 1,363,292	6.16 / 77.8
Railcab / R2	97,522 / 317,094	437,383 / 2,842,593	15.5 / 283

L: Lazy; DF: Direct-Full. All 14 WIN/WIN pairs are shown. States and successor-oracle queries are exact first-trial counts. Solver seconds are medians of five valid runs, including internal checking and excluding JVM/frontend/endpoint time, rounded to three significant digits.

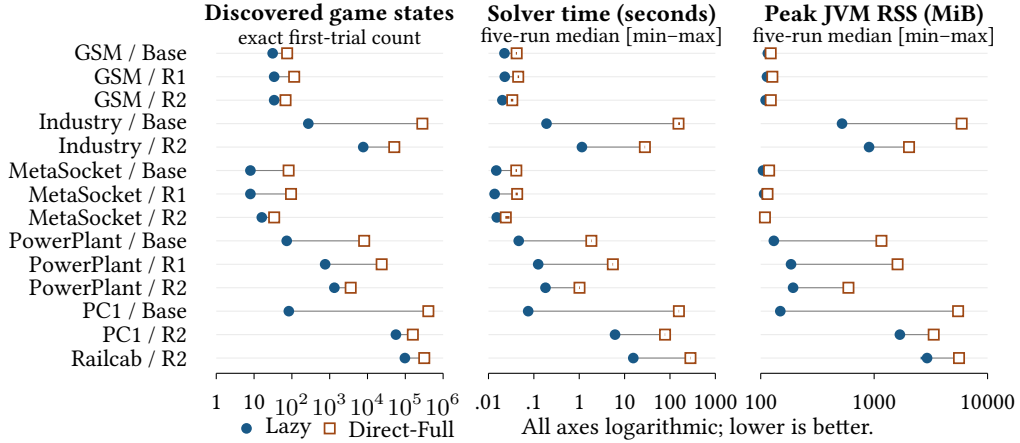


Figure A4: All 14 completed WIN/WIN pairs. Solver time includes checks and excludes frontend, endpoint preparation and JVM startup. Resident set size (RSS) is sampled over whole JVM runs and does not measure heap use. The other 13 Direct-Full contracts time out and are omitted (Figure A5). This compares procedures with different UC pruning.

O Separate Extended-Budget Trials

Extended-budget trials. Table A8 separates single-trial populations on the same Xeon/JAR: 5 comparable Direct-Full decisions match fixed-budget Lazy; 3 Travel (4, 8) WIN decisions resolve prior timeout. Direct-Full timeouts, Surveillance/R1 heap exhaustion and Lazy PC2/R2 timeout persist at expanded heaps. The shared Travel (6, 1) timeout triggers the prespecified (6, 4) skip (records in S4).

Table A8: Separate enlarged-budget single trials (7,200 s cap). S4 retains every run, failure and skip.

Population	Heap (GiB)	Planned	WIN	LOSS	TO	OOM	Skip
Earlier (four methods)	64/200	27	7	1	16	1	2
Eager	200	10	3	1	5	1	—

On the same Xeon/JAR, Eager’s 200 GiB/7,200 s single trials return 3 WIN where Direct-Full remained unresolved at that heap, discovering about $(1.08\text{--}2.66) \times 10^7$ states with UC pruning (60–68,000 times Lazy); Industry/R1 returns LOSS with 94,046 states (2.4 times). All 4 decisions match fixed-budget Lazy; ratios concern decided pairs only. The other 6 trials yield 5 TO/1 OOM at about 209–212 GiB peak RSS, a process-memory measure (S4).

P Separate Evaluation Populations and All-Contract Grid

P.1 Validation populations and guarantee scopes

Table A9 details the populations in main-paper Table 2. Source-monitor interpretation does not establish actual activation conformance.

Table A9: RQ2 validation populations, keyed to the scopes in main-paper Table 2.

Scope and check	Population	Established result and boundary
Game: independent games	46 fixtures, two solvers: 92 jobs	36 WIN, 48 LOSS, 8 invalid; all 84 eligible jobs pass certificate/exhaustive checks.
Invariant from start: New-requirement lookup tables	46 source positions, 138 occurrences; 900 non-error cells	Exact start-scoped languages under reference observations (Proposition F.1); general activation conformance remains a premise.
Invariant from start: Saved Railcab policies	Base/R1: 22 entries each, 21 start edges; shared post graph	Source-observer replay and truth checks on all saved continuations; accepts exported entry vectors, excludes runtime/fallback (Appendix F.3).
From update entry: Interval initialization	135 occurrences: 46 R1, 89 R2	Exact entry-scoped initialization under reference observations; adapter conformance unchecked.
Game: application policies	25 Lazy WIN contracts; 125 runs	Certificates check the shared Java game. Runs save summaries; complete PC2-Rolling/Railcab exports are separate.

The following descriptions preserve the separate secondary populations reported in the main paper. They are not pooled with the fixed-budget comparisons. Figure A5 displays all 108 fixed-budget outcomes. Figure A4 gives absolute costs for every completed Lazy/Direct-Full pair; main-paper Figure 3 compares Lazy with Eager under the same UC pruning.

	Lazy			Update-first			Eager			Direct-Full		
Adapted input	B	R1	R2	B	R1	R2	B	R1	R2	B	R1	R2
GSM	W	W	W	W	W	W	W	W	W	W	W	W
Industry	W	L	W	W	L	W	W	T	W	W	T	W
MetaSocket	W	W	W	W	W	W	W	W	W	W	W	W
PowerPlant	W	W	W	W	W	W	W	W	W	W	W	W
PC1	W	W	W	W	W	W	W	W	W	W	T	W
PC2	W	W	T	T	T	T	T	T	T	T	T	T
Railcab	W	W	W	W	W	W	T	T	W	T	T	W
Surveillance	W	W	W	W	W	W	T	T	T	T	T	T
Workflow	W	W	W	W	W	W	W	T	W	T	T	T

W: WIN L: LOSS T: timeout B: Base; R1/R2: interval variants

Figure A5: Every fixed-budget contract–method outcome. Each tile is one of the same 9 inputs $\times$ three requirement variants, under a 1,200 s cap and 64 GiB heap. WIN and LOSS are completed decisions; timeout leaves feasibility unresolved. The methods change exploration, keeping entries, successors and goals fixed. The comparison evaluates complete exploration procedures. Workflow/R2’s Direct-Full tile uses the recorded same-setting timeout after interruption; both attempts are retained.

Base/R1/R2’s 5/3/6 pairs give median state ratios 111.89/11.88/2.75 and solver ratios 40.02/3.12/9.14. The largest state savings are in Base, but different completed populations do not isolate interval requirements’ effect.

Order sensitivity in R2. Eight R2 contracts complete all five repetitions with both Lazy and Update-first; PC2/R2 times out with both. Update-first discovers fewer states and has a lower solver median on Industry, PowerPlant, PC1, Railcab, Surveillance and Workflow; Lazy is lower on both measures for GSM and MetaSocket. For example, PC1/R2 has 56,443 versus 9,304 discovered states and 6.155 versus 1.166 seconds for Lazy versus Update-first. These saved results expose a substantial order dependence, but do not isolate a causal interaction between the order and interval monitors. The performance contribution concerns the full lazy procedure relative to construction alternatives, not a uniformly best exploration order.

P.2 Complete records for the separate populations

Supplement Sections S4.5.3 and S4.5.4 retain all Full+UC, restricted-transfer, controlled-family and Travel outcomes. Full+UC mixes builds, hosts and repetitions; its timing does not isolate UC pruning. For Legacy’s subsequent-control reference, fixed-budget OOM/unmeasured cells are in Supplement Section S4.3, source/fork discrepancies in Section S4.8, and separate enlarged-heap trials in Section S4.9. Other enlarged-budget failures and skips are in Appendix O and Supplement Section S4.6; none enters fixed-budget medians.

The boundary fixture distinguishes forbidding an intervening event from merely merging same-type boundary labels (Supplement Section S4.5.2); Cell_n and restricted-transfer controls follow in Section S4.5.3. The original finite-file results remain in Section S4.2; Section S4.5.3 retains earlier failed inputs. Supplement Sections S4.4.2 and S4.4.3 give the complete Industry losing-region and Railcab policy diagnoses. Industry’s global losing closure excludes every order; the validation-permitting variant changes the contract and supplies no timing data. The native DUCS WIN uses a different subsequent-control objective (main-paper Section 6.3).

P.3 Input adaptation details

Nine inputs manually adapt eight DUCS sources; both ProductionCell sizes share a parent. Base/R1/R2 vary interval requirements, fixing components, transfers, endpoints and old/new goals (Technical Appendix M). Adaptations make PC tool results UC, revise Surveillance battery behavior/safety, order GSM/R2’s decode boundaries before ENV transfer, shorten Railcab transfer paths while keeping both

outcomes, and omit PowerPlant’s explicit ENV_OLD-to-ENV_NEW transfer declaration while retaining STARTED-to-STARTED; the environment names alias those initial states. Recorded provenance does not establish whole-problem equivalence.

PC1/PC2 retain their DUCS parent’s repeatable arrival cycle: UC in_i enters an occupied arm; controllable out_i and UC $reset_i$ permit its next arrival. With work and output withheld, pending tool results or reset/arrival chains finish in quiescent arm states. PC2-Rolling adds one UC calibration-completion step. This is finite drainage of pending physical events, not a total arrival budget or a safety proof. DUCS’s production-cell study likewise distinguishes an empty-line condition from delaying newly arrived raw work [31, Sec. 6.1]. FG commands require no enabled UC event; a temporal gap between arrivals alone does not ensure this.

Travel couples an agency with N amenities, K UC choices and $0-K-1$ selection steps: size, branching and length covary. Adaptations change purchasing, admission, transfers and monitors without claimed trace equivalence. Supplement Section S4.5.4 retains every setting and completed-pair comparison; $(N, K) = (4, 4), (4, 6)$ are the only settings extending OTF’s fixed-budget decided range.

Independent-update construction. The complete $n+1$ versus 2^n construction and its proof occur once, in Supplement Section S2.3. It assumes the measured pending-transfer-first query order and supplies no order-independent performance guarantee.

P.4 Memory and returned policies

Lazy’s five-run median peak working sets span 0.102–7.36 GiB (median 0.837), including JVM/native overhead; Eager/Update-first sometimes use less. Its 25 WIN certificates contain 8–1,071 update states (median 55) and maximum ranks 4–34 (median 19). Ranks bound events, not downtime or optimal length. Counts match all five repeats; LOSS/TO has no winning policy.

The rank domain is recorded as `certificate_states` in saved `rq3/stage1.csv`. S4’s “Output policy states” instead uses `output_controller_states`: the linked old–update–new controller, with 15–1,260 states for Lazy. Linking includes reachable old and new closed-loop states and non-goal update states, identifying each update goal with its selected new state. These columns describe the same successful runs at different construction stages; neither counts explored game states.

P.5 Lifetime gaps in two saved policies

The separate qualitative Railcab Base/R1 exports in S4 retain their full policy graphs. Define a gap on a completed path as the labels strictly after its last old-stop edge and strictly before its first new-start edge, excluding both boundaries. A normal label is outside the complete command set pending at entry. All 22 entries of each export have every old stop before the first new start on every possible policy path. There are 34 Base and 41 R1 completed paths when paths from different old entries are counted separately.

Saved policy	States / edges	Entries	Gap labels	Ordinary gap labels
Railcab Base	115 / 118	22	5–8	1–4
Railcab R1	44 / 46	22	3	0

The Base policy exploits a period without old or new requirements; R1 keeps its seven interval monitors active even when those requirements are inactive. The observed zero ordinary events in R1 describes this selected policy, not every winning policy. These counts quantify events, not elapsed time or suppressed alternatives.

The revised source’s `scripts/analyze_saved_railcab_gap.py` traverses every saved outcome from every entry. It verifies entry coverage, strict rank decrease, one-shot command consumption, active-monitor membership, full certificate reachability and all terminal paths. Its inputs are the two `handoff-bundle.json` files under `results/supplement-checks/ra`

ilcab_policy_demo/ and results/supplement-checks/railcab_r1_policy_demo/; it returns per-entry ranges and extremal path segments. Counting internal edges of the region with no active old/new monitor independently gives the same ranges for all 44 entries. The input bundles and this analysis script are included in the matching public snapshot.

This is a static analysis of the already saved qualitative exports, with no new synthesis or timing trial. Lazy’s 125 successful fixed-budget repetitions instead retain summary-only controller output, certificate sizes and maximum ranks. Their unrecorded policy edges cannot establish corresponding gap ranges for all 25 WIN contracts. Neither these two exports nor their analysis reconstructs enabled but unselected events or an optimal completion rank.

Reading separate performance populations. Supplement Section S4.4.1 defines the capped-attempt JVM lower bounds and retains the full metric panels. They are neither solver ratios nor medians of censored trials. Auxiliary and extended populations remain separate from the fixed-budget comparison; their complete records are linked above.

Applicability to inherited transfer comparisons. We inspected all nine saved FSP inputs and their Base/R1 transfer comparisons. Table A10 records the equal-alphabet specialization of the transfer condition, including failures. This is source inspection, not an independently executed FSP semantic checker or new synthesis experiment. The artifact archives source excerpts and a record-consistency checker in its serial-transfer-inputs result directory.

Table A10: Equal-alphabet transfer-condition inspection for every inherited input.

Input	Same UC alphabet	No newly enabled UC
GSM, Industry, MetaSocket	yes	yes
PowerPlant, Surveillance, Workflow	yes	yes
PC1, PC2	no	yes
Railcab	no	no

The six positive inputs have empty UC sets, unchanged local graphs with state-preserving transfers, or changes confined to controllable behavior. PC1/PC2 replace UC polishing-result labels with painting-result labels, violating alphabet equality. Their ARM-to-ARM transfer pairs also fail the generalized condition: both sides enable no UC event, but removing the blocked polishOk and polishNok labels violates $B_i^{\text{old}}(s) \subseteq B_i^{\text{new}}(s')$. Railcab adds UC approachingCrossing and enables it through a declared transfer from old to new ENDOFTS; both conditions fail.

All 138 Base/R1 NEW/UPD monitor occurrences omit transfer labels; OLD source requirements mention ordinary labels only. Alphabet-external labels stutter, so these transfer effects commute. All 18 saved contracts have no explicit precedence. The retained merge transformation preserves the other comparison premises and validates commutation. This supports applicability to 12/18 transfer-only comparisons: 11 observed WIN/WIN pairs and Industry/R1’s LOSS/LOSS pair, where the antecedent is false. The other six pairs are WIN/WIN despite failing this equal-alphabet specialization. The 36 boundary/both comparisons and R2 applicability are excluded. The implication is merged WIN $\Rightarrow$ fine WIN; it does not explain the absence of fine WIN/merged LOSS in the 54 comparisons. Compiled OLD testers were not separately exported; this analysis establishes source/model premises, not runtime conformance.

Finite checks of the transfer condition. A saved auxiliary check of the equal-alphabet specialization considers 4,352 two-component games and 16,640 all-old starting tuples: 256 exhaustive one-state cases and 4,096 seeded two-state cases. The latter are the first 4,096 of 10,695 generated candidates having a nonempty quiescent all-new target set; 6,599 outside that domain are retained as excluded counts. The check uses ordinary controls, UC events, nondeterministic outcomes, partial transfers and arbitrary nonempty target subsets. It finds 1,910 both-WIN starts, 63 fine-only WIN starts and 14,667 both-LOSS

starts, with no coarse-only WIN. It also checks 6,541 partial-prefix quiescence obligations and 5,124 Cartesian outcome comparisons. Monitors and precedence are absent and endpoint controllers are not reconstructed, so this does not test the commuting-monitor, precedence or frontend premises. Seed 20261003 and both failed-condition counterexamples are retained with the checker. This is a finite sanity check of the stated implication, not its proof, an application benchmark or a performance result.

Q Implementation and AI-Assisted Research Details

Algorithm 1 returns and internally checks the losing region L . The finite-JSON runner serializes its members in `certificate.json`; an independent checker reconstructs successors from the input and checks closure. The FSP/application benchmark driver instead records LOSS, region size and phase-mask statistics, without serializing solver membership. Its saved full-game exports support independent region reconstruction, but do not compare the solver’s member list. These output interfaces therefore provide different saved certificate evidence. The fixed RQ3 runs use Windows 10, an Intel Xeon W-2265, 256 GB RAM and Temurin 17.0.20.1. The main paper states the heap, time cap and repetition policy; separate Mac structural checks and extended trials retain their own provenance.

Codex and Claude assisted the research in the following roles. For design and theory, these tools proposed contract definitions, examples, counterexamples and proof drafts, which were checked by explicit derivations and finite-model checks where provided. For implementation, they edited the Java frontend, successor generation, solver, certificate checks and Python reconstruction tools; regression fixtures, exhaustive comparisons and certificate checks supplied validation. For experiments, they helped build and run drivers and aggregate saved outputs; the tables retain capped, failed and adverse outcomes, and reproduction scripts check their correspondence to saved results. For analysis, they compared source formulas, monitor automata, initializer tables and saved policies within each stated check’s scope. For writing, they drafted and edited text, diagrams and references; cited claims were checked against primary sources. These checks are evidence about their specified objects, not independent proof of all AI-produced code or reasoning.

R Static Expansion of Saved Threads Policies

This analysis transforms previously saved winning policies; it performs no synthesis or timing measurement. The cohort is all ten saved Threads backpressure instances with $n = 2, \dots, 6$ and queue capacity $B \in \{1, 2\}$. Saturated-offer cases retain their earlier LOSS results (Table 3). The forty copied input, policy and result files are byte-identical to their counterparts in the cited archive’s tag `fgducS-c115-20261003`. The new script and outputs accompany this manuscript’s source bundle, not that earlier release.

Construction and verification. The transformer copies every ordinary edge and replaces the saved policy’s single simultaneous transfer edge by $\rho_1, \dots, \rho_n$. Its $n - 1$ added states record the saved checkpoint, target and remaining block members. Non-goal saved ranks increase by $n - 1$; the added phases have ranks $n - 1, \dots, 1$, followed by the saved rank-zero goal. Separate verification functions reconstruct the synchronous successor relation from the copied local LTSs and check all old-endpoint entries, every enabled UC action, every selected action’s complete outcomes, decreasing ranks, reachability and non-goal continuation. They also reconstruct the reachable old/new plant-controller products and compare their full projected transition relations with the declared endpoints. Each transformed goal must match the same saved endpoint and loadable target.

Contracting each forced serial chain recovers the saved ordinary graph. No UC action is enabled at any inserted phase. Consequently corresponding complete paths have exactly $n - 1$ additional update events. A separate acyclic-graph pass checks complete-path length bounds and transfer counts from every entry. This is an event-count statement, not a time or optimality result.

Table A11: All saved-policy expansions. S/E denotes states/edges; ranks are maximum entry ranks. Every row passes all checks.

n	B	Entries	Saved S/E	Expanded S/E	Saved rank	New rank	Extra events
2	1	8	9/13	10/14	4	5	1
2	2	12	13/21	14/22	5	6	1
3	1	16	17/33	19/35	5	7	2
3	2	24	25/53	27/55	6	8	2
4	1	32	33/81	36/84	6	9	3
4	2	48	49/129	52/132	7	10	3
5	1	64	65/193	69/197	7	11	4
5	2	96	97/305	101/309	8	12	4
6	1	128	129/449	134/454	8	13	5
6	2	192	193/705	198/710	9	14	5

All ten cases pass, with no failed case. Across related instances, 630 saved states and 1,982 edges become 660 states and 2,012 edges. The checks cover 620 entry occurrences, 1,972 UC action buckets/outcomes and 2,012 selected buckets/outcomes. Every bucket in this cohort has one outcome. Five corruptions per case—omitting a UC outcome or entry, corrupting a rank or transfer outcome, and removing a loadable endpoint—are all rejected (50/50). A rerun produces byte-identical outputs.

Scope and reproduction. These are related finite abstractions, not independent applications. OLD and NEW local LTSs are identical; transfers are deterministic identities at idle states; requirement and precedence sets are empty. Thus the check does not exercise nondeterministic transfer outcomes, changing local behavior, monitors, initializer boundaries or nontrivial UC interleavings during serialization. It does not convert the twelve inherited input pairs or change any earlier WIN/LOSS result. The Python checker is separate from the Java solver and earlier checker, but has the same authors and AI assistance; correlated mistakes remain possible. The source bundle includes reproduction instructions and all inputs/results in `evidence/threads_saved_serialization/`.